```
// ArrowRushSim.cs
using System;
using System.Collections.Generic;
namespace Sy.G06
{
    /// &lt;summary&gt;
    /// E4 graybox sim for ArrowRush (G06 candidate, timing-shoot archery stack).
    /// Pure C#, zero UnityEngine dependency: headless EditMode tests drive the clock.
    /// Core loop: hold to charge -&gt; release at the right beat -&gt; arrow lands and STACKS
    /// onto the tower; bad beats shake stability; collapse offers a revive (hybrid seat 1),
    /// perfect beats drop shards (hybrid collect seat 2).
    /// E3 conditions absorbed as asserts (E4 gate):
    ///   1) stack-arrow hook perceivable within 3s of first landing;
    ///   2) hybrid monetization seats exist in the loop (fail offer + collection).
    /// &lt;/summary&gt;
    public sealed class ArrowRushSim
    {
        // Timing verdict bands (charge power 0..1, oscillating meter).
        public const float PerfectMin = 0.85f;
        public const float PerfectMax = 0.98f;
        public const float GoodMin = 0.60f;
        // Meter sweep speed in units/sec (E4 GDD sec 2 anchor, 1.2 cycles/sec).
        // Single source of truth: the catalog difficulty stage 0 pins against it.
        public const float ChargeSweepSpeed = 2.4f;
        // Collapse tuning: stability in [-4, +4]; each miss -2, each good +0.5, perfect +1.
        public const float StabilityMax = 4f;
        public const int MaxCollapses = 3;
        // S2-T003 slice A: full-power split (pitch mechanism 2, GAP-B bot-pinned).
        // Full power = the meter-top clamp frame (charge 1.0): at any frame rate the
        // only reachable value above PerfectMax, so the trigger is frame-rate independent.
        public const float FullPowerCharge = 1f;
        public const int SplitArrowCount = 3;
        public const float SplitHeightFraction = 0.5f; // fork at half the flight (View slice B consumes)
        public const float GoodStabilityBonus = 0.5f;  // mirrors the Good case; greed gets no perfect bonus
        // S2-T004 slice A: wind hourglass (pitch mechanism 3, GAP-B second half).
        // Opt-in additive layer, default OFF - frozen E4 paths never read it. Wind
        // alternates Tail -&gt; Head every WindPeriodSeconds; headwind quickens the
        // charge sweep (bot band: 1.25 no effect, 1.5 exactly one verdict tier down
        // for the fixed-hold bot, 1.75+ miss cascade); releasing inside the catch
        // window right after a head-&gt;tail flip grants a perfect-tier stability shield.
        public const float WindPeriodSeconds = 10f;      // pitch anchor; a rhythm event at bot cadence
        public const float HeadwindSweepMul = 1.5f;      // band center: one tier down, no collapse cascade
        public const float WindCatchWindow = 1.2f;       // &gt;= one 0.675s bot cadence slot, &lt;= one casual 2s slot
        public const float WindCatchStabilityBonus = 1f; // perfect-tier shield: caught miss -2 +1 nets -1
        public enum Beat { None, Perfect, Good, Miss }
        public enum Phase { Opening, Playing, Failed }
        public sealed class SimEvent
        {
            public float SimSeconds;
            public string Kind;
```

```
    public string Detail;
    public SimEvent(float t, string kind, string detail)
    {
        SimSeconds = t;
        Kind = kind;
        Detail = detail;
    }
}
readonly List&lt;SimEvent&gt; _events = new List&lt;SimEvent&gt;();
float _clock;
float _charge;          // oscillating charge meter while held
float _chargeDir = 1f;
bool _holding;
float _stability = 2f;
int _collapseCount;
int _shots;
int _perfectCount;
int _shardCount;
bool _reviveOfferShown;
bool _guidePromptShown;
bool _firstWowFired;
int _splitShots;
// S2-T004 slice A wind state (additive; nothing frozen reads these).
bool _windEnabled;
int _windDir = 1;            // +1 tail, -1 head; the hourglass opens tail (teaching-friendly)
float _windTimer;
float _tailFlipArm = -1e9f;  // clock of the last head-&gt;tail flip; arms the catch window
public Phase CurrentPhase { get; private set; } = Phase.Opening;
public int StackHeight { get; private set; }
public IReadOnlyList&lt;SimEvent&gt; Events =&gt; _events;
public int Shots =&gt; _shots;
public int PerfectCount =&gt; _perfectCount;
public int ShardCount =&gt; _shardCount;
public int CollapseCount =&gt; _collapseCount;
public bool ReviveOfferShown =&gt; _reviveOfferShown;
public float Stability =&gt; _stability;
/// &lt;summary&gt;
/// Count of full-power shots that split into 3 arrows (S2-T003 slice A,
/// additive state - no frozen path reads or writes it).
/// &lt;/summary&gt;
public int SplitShots =&gt; _splitShots;
/// &lt;summary&gt;
/// Live charge meter value 0..1 (S1-T001 additive read-only accessor for the
/// presenter arc bar; behavior untouched - the E4 13-assert tree stays frozen).
/// &lt;/summary&gt;
public float Charge =&gt; _charge;
/// &lt;summary&gt;
/// S2-T004 slice A wind read face for the presenter/View (slice B hourglass HUD).
/// WindDirection: +1 tail / -1 head; WindPhaseRemaining: seconds to the next
/// flip; WindCatchOpen: the catch window after a head-&gt;tail flip is live now.
```

```
/// &lt;/summary&gt;
public bool WindEnabled =&gt; _windEnabled;
public int WindDirection =&gt; _windDir;
public float WindPhaseRemaining =&gt; WindPeriodSeconds - _windTimer;
public bool WindCatchOpen =&gt;
    _windEnabled &amp;&amp; _windDir &gt; 0 &amp;&amp; _clock - _tailFlipArm &lt;= WindCatchWindow;
/// &lt;summary&gt;Hook caption lands instantly at t=0 (red line 11: no white-screen teaching).&lt;/summary&gt;
public void Open()
{
    Emit("caption", "Stack arrows into a tower!");
    CurrentPhase = Phase.Playing;
}
/// &lt;summary&gt;First-time guide prompt fires as soon as play opens (&lt;=3s budget in tests).&lt;/summary&gt
public void MaybeShowGuide()
{
    if (_guidePromptShown) return;
    _guidePromptShown = true;
    Emit("guide_prompt", "hold and release on the beat");
}
/// &lt;summary&gt;Advance the sim clock by dt seconds. Opt-in wind hourglass flips on
/// period boundaries (processed before the sweep so a boundary-spanning dt uses
/// the post-flip direction); the charge meter oscillates while holding and
/// headwind quickens the sweep by the bot-pinned multiplier.&lt;/summary&gt;
public void AdvanceClock(float dt)
{
    _clock += dt;
    if (_windEnabled)
    {
        _windTimer += dt;
        while (_windTimer &gt;= WindPeriodSeconds)
        {
            _windTimer -= WindPeriodSeconds;
            _windDir = -_windDir;
            if (_windDir &gt; 0) _tailFlipArm = _clock; // head -&gt; tail flip arms the window
            Emit("wind_shift", _windDir &gt; 0 ? "dir=tail" : "dir=head");
        }
    }
    if (!_holding) return;
    // Charge sweeps 0..1 at the E4-anchored speed: timing skill = release near the top.
    float speed = (!_windEnabled || _windDir &gt; 0) ? ChargeSweepSpeed : ChargeSweepSpeed * HeadwindSweepMul;
    _charge += _chargeDir * speed * dt;
    if (_charge &gt;= 1f) { _charge = 1f; _chargeDir = -1f; }
    else if (_charge &lt;= 0f) { _charge = 0f; _chargeDir = 1f; }
}
public void BeginHold()
{
    _holding = true;
    _charge = 0f;
    _chargeDir = 1f;
}
```

```
/// &lt;summary&gt;Release the shot at the current charge value; resolves beat, stacking, stability.&lt;/summary
public Beat ReleaseShot()
{
    if (!_holding) return Beat.None;
    _holding = false;
    _shots++;
    var beat = Classify(_charge);
    // Feedback latency contract: verdict event lands the same tick (&lt;=1s budget in tests).
    Emit("shot_released", $"beat={beat} power={_charge:0.00} shot={_shots}");
    // S2-T003 slice A (additive): full-power split. The meter-top clamp frame is
    // the trigger; the fork happens mid-flight, strictly before the landing
    // events. Verdict vocabulary stays frozen (Classify untouched) - the split is
    // a resolution identity layered on the Good verdict, not a fourth beat.
    bool fullPower = _charge &gt;= FullPowerCharge;
    if (fullPower)
    {
        _splitShots++;
        Emit("arrow_split", $"splits={SplitArrowCount} fraction={SplitHeightFraction:0.00}");
    }
    Emit("arrow_landed", $"beat={beat} shot={_shots}");
    if (fullPower)
    {
        // Landing resolution (GAP-B bot-pinned): 3 stack rows from one shot,
        // good-tier stability only (greed wobbles the tower, never the perfect
        // bonus) and zero shards - the perfect-only shard economy stays frozen.
        _stability = Math.Min(StabilityMax, _stability + GoodStabilityBonus);
        for (int i = 0; i &lt; SplitArrowCount; i++)
        {
            StackHeight++;
            if (i == 0) Emit("stack_signal", $"height={StackHeight}");
            else Emit("split_stack", $"height={StackHeight} arrow={i + 1}/{SplitArrowCount}");
        }
    }
    else
    {
        switch (beat)
        {
            case Beat.Perfect:
                _perfectCount++;
                _stability = Math.Min(StabilityMax, _stability + 1f);
                StackHeight++;
                Emit("stack_signal", $"height={StackHeight}");
                _shardCount++;
                Emit("shard_drop", $"shards={_shardCount}");
                break;
            case Beat.Good:
                _stability = Math.Min(StabilityMax, _stability + 0.5f);
                StackHeight++;
                Emit("stack_signal", $"height={StackHeight}");
                break;
```

```
                default:
                    _stability -= 2f;
                    Emit("arrow_dropped", $"stability={_stability:0.0}");
                    if (_stability &lt;= 0f) Collapse();
                    break;
            }
        }
        // S2-T004 slice A (additive): wind-catch. Releasing inside the window right
        // after a head-&gt;tail flip rides the gust: a perfect-tier stability shield on
        // any beat (a caught miss nets -1 - the wind softens, never rewrites, the
        // verdict vocabulary; shard and split resolutions stay frozen).
        if (_windEnabled &amp;&amp; _windDir &gt; 0 &amp;&amp; _clock - _tailFlipArm &lt;= WindCatchWindow)
        {
            _stability = Math.Min(StabilityMax, _stability + WindCatchStabilityBonus);
            Emit("wind_catch", $"stability={_stability:0.0}");
        }
        // First-wow moment: tower visibly tall for the first time (share/boast seed).
        if (!_firstWowFired &amp;&amp; StackHeight &gt;= 3)
        {
            _firstWowFired = true;
            Emit("first_wow", $"height={StackHeight}");
        }
        return beat;
    }
    void Collapse()
    {
        _collapseCount++;
        StackHeight = 0;
        _stability = 1.5f;
        Emit("tower_collapse", $"collapse={_collapseCount}");
        if (_collapseCount &gt;= MaxCollapses)
        {
            CurrentPhase = Phase.Failed;
            _reviveOfferShown = true;
            // Hybrid seat 1: fail-state revive offer (near-miss "one more shot" tension).
            Emit("revive_offer", "watch to restore tower");
        }
    }
    public static Beat Classify(float power)
    {
        if (power &gt;= PerfectMin &amp;&amp; power &lt;= PerfectMax) return Beat.Perfect;
        if (power &gt;= GoodMin) return Beat.Good;
        return Beat.Miss;
    }
    /// &lt;summary&gt;Reset after revive: back to play with a fresh tower (hybrid seat 1 consumer).&lt;/summary&gt;
    public void AcceptRevive()
    {
        if (CurrentPhase != Phase.Failed) return;
        CurrentPhase = Phase.Playing;
        _collapseCount = 0;
```

```
            _stability = 2f;
            Emit("revive_accepted", "tower restored");
        }
        /// &lt;summary&gt;
        /// Opt in to the wind hourglass (S2-T004 slice A). Frozen E4 paths never call
        /// this. The layer is additive: the hourglass opens tail, flips every period,
        /// and the catch window arms only on actual head-&gt;tail flips - the enable
        /// moment is a phase start, not a reversal, so nothing is free at boot.
        /// &lt;/summary&gt;
        public void EnableWind()
        {
            _windEnabled = true;
            _windDir = 1;
            _windTimer = 0f;
        }
        void Emit(string kind, string detail)
        {
            _events.Add(new SimEvent(_clock, kind, detail));
        }
        /// &lt;summary&gt;Sim-seconds timestamp of the first event of the given kind, or -1 if absent.&lt;/summary&gt;
        public float FirstEventTime(string kind)
        {
            for (int i = 0; i &lt; _events.Count; i++)
                if (_events[i].Kind == kind) return _events[i].SimSeconds;
            return -1f;
        }
        /// &lt;summary&gt;Count of events of the given kind.&lt;/summary&gt;
        public int CountEvents(string kind)
        {
            int n = 0;
            for (int i = 0; i &lt; _events.Count; i++)
                if (_events[i].Kind == kind) n++;
            return n;
        }
    }
}
// G06AdFunnel.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.G06.Boot
{
    /// &lt;summary&gt;
    /// evt_ad funnel for G06 (task S2-T002, P02 T015 eight-point paradigm). The
    /// three-piece law every offer face consumes: (1) CanOffer pre-checks the point's
    /// category budget on the SAME policy instance the suite's ads service consumes
    /// from - an over-cap offer never fires, no dead ad buttons; (2) the accept flow
    /// goes through the category-aware ShowRewarded(pointId) so the daily caps read
```

```
/// exactly as the design table (rescue 4 / double 5 / direct 1, global 12);
/// (3) every state transition lands as an evt_ad telemetry event
/// (offer/accept/blocked/granted/ad_fail + point + reason) so the funnel is
/// auditable end to end.
/// &lt;/summary&gt;
public sealed class G06AdFunnel
{
    public const string StateOffer = "offer";
    public const string StateAccept = "accept";
    public const string StateAdFail = "ad_fail";
    public const string StateBlocked = "blocked";
    public const string StateGranted = "granted";
    readonly TelemetryService _telemetry;
    readonly IAdsService _ads;
    readonly AdCapPolicy _policy;
    readonly Func&lt;string&gt; _dayKey;
    public G06AdFunnel(TelemetryService telemetry, IAdsService ads, AdCapPolicy policy, Func&lt;string&gt; dayKey =
    {
        _telemetry = telemetry;
        _ads = ads;
        _policy = policy;
        _dayKey = dayKey;
    }
    /// &lt;summary&gt;Pre-offer gate: true when the point's category still has budget
    /// today (uncategorized points always pass). Null policy face = legacy
    /// ungated behavior, never a silent block.&lt;/summary&gt;
    public bool CanOffer(string pointId)
    {
        if (_policy == null) return true;
        string category = AdCategories.CategoryOfPoint(pointId);
        string key = _dayKey != null ? _dayKey() : AdCapPolicy.TodayKey();
        return _policy.HasCategoryBudget(key, category);
    }
    /// &lt;summary&gt;Accept flow entry: fires evt_ad accept, then shows the rewarded
    /// view through the category-aware service. Callers await the returned task
    /// and report granted/ad_fail from the result - an ad reward is never
    /// granted silently and never lost silently.&lt;/summary&gt;
    public Task&lt;RewardResult&gt; Accept(string pointId, string reason)
    {
        Track(StateAccept, pointId, reason);
        return _ads.ShowRewarded(pointId);
    }
    public void ReportOffer(string pointId, string reason) =&gt; Track(StateOffer, pointId, reason);
    public void ReportBlocked(string pointId, string reason) =&gt; Track(StateBlocked, pointId, reason);
    public void ReportGranted(string pointId, string reason) =&gt; Track(StateGranted, pointId, reason);
    public void ReportAdFail(string pointId, string reason) =&gt; Track(StateAdFail, pointId, reason);
    void Track(string state, string pointId, string reason)
    {
        _telemetry?.Track(EventIds.Ad, new Dictionary&lt;string, string&gt;
        {
```

```
                { "state", state },
                { "point", pointId ?? "" },
                { "reason", reason ?? "" },
            });
        }
    }
}
// G06AdPoints.cs
namespace Sy.G06.Boot
{
    /// &lt;summary&gt;
    /// G06 rewarded point ids (task S2-T002). Namespaced with the g06_ prefix so no
    /// G06 call can ever draw P02/P06/G02/G03/G04/G05 category budget and vice versa.
    /// Category mapping lives in Sy.Platform.AdCategories.CategoryOfPoint: Revive = the
    /// fail-state revive offer seat (S1-built, rescue 4/day), ShardDouble = the shard
    /// settle doubling offer (double 5/day), SkinPick = the arrow-skin direct pick
    /// (direct 1/day, consumed by the S2-T005 collection album), all under the 12/day
    /// global cap.
    /// &lt;/summary&gt;
    public static class G06AdPoints
    {
        public const string Revive = "g06_revive";
        public const string ShardDouble = "g06_shard_double";
        public const string SkinPick = "g06_skin_pick";
    }
}
// G06BootServices.cs
using Sy.Core;
using Sy.Platform;
using Sy.Telemetry;
using Logger = Sy.Core.Logger;
namespace Sy.G06.Boot
{
    /// &lt;summary&gt;
    /// Platform suite wiring for G06 (task S2-T002, G02/G03 BootServices paradigm).
    /// PlatformBootstrap.Create(policy) resolves editor/SY_MOCK to the mock suite and
    /// real WeChat builds to the Wx suite, so the editor path and a production build
    /// differ only by define, never by wiring. The SAME AdCapPolicy instance is exposed
    /// as SharedPolicy so offer faces pre-check budgets (HasCategoryBudget) on the exact
    /// policy the suite's ads service consumes from - pre-check and consumption can
    /// never disagree (P02 T006-B2 pattern). Zero-server red line 12: the TCB batch
    /// sink class stays available in the shared package but is NEVER registered here.
    /// &lt;/summary&gt;
    public static class G06BootServices
    {
        /// &lt;summary&gt;The policy the registered suite consumes from; null before RegisterAll.&lt;/summary&gt;
        public static AdCapPolicy SharedPolicy { get; private set; }
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;IAdsService&gt;(out _)) return; // idempotent: double register would throw
```

```
            var policy = new AdCapPolicy();
            SharedPolicy = policy;
            var suite = PlatformBootstrap.Create(policy);
            ServiceLocator.Register&lt;IAdsService&gt;(suite.Ads);
            ServiceLocator.Register&lt;IPlatformService&gt;(suite.Platform);
            ServiceLocator.Register&lt;IStorage&gt;(suite.Storage);
            // Launch-entry query face joins the locator so a boot host can read the
            // share attribution exactly once (ILaunchInfo contract).
            ServiceLocator.Register&lt;ILaunchInfo&gt;(suite.Launch);
#if SY_WECHAT &amp;&amp; !UNITY_EDITOR
            // Wx custom analytics joins the telemetry pipeline via
            // G06TelemetryWire.CreateService; mock/editor builds compile this out
            // entirely and the debug sink stays the only one.
            ServiceLocator.Register&lt;IAnalyticsReporter&gt;(new G06WxAnalyticsReporter());
#endif
            Logger.Info("BOOT", "g06 platform suite registered (suite=" + PlatformBootstrap.SuiteIdFor(suite.Channel)
                + ", g06 ad categories: rescue 4 / double 5 / direct 1, global 12/day)");
        }
    }
}
// G06TelemetryWire.cs
using Sy.Core;
using Sy.Telemetry;
namespace Sy.G06.Boot
{
    /// &lt;summary&gt;
    /// Telemetry assembly point for G06 (task S2-T002, G03TelemetryWire paradigm):
    /// DebugLogSink is always attached; the Wx analytics sink joins when a reporter
    /// is available (explicit param wins, otherwise the locator is consulted for
    /// the platform-registered Wx reporter on real builds). Zero-server red line 12:
    /// the TCB batch sink class stays available in the shared package but is never
    /// registered. Event schema faces (ad funnel) live in G06AdFunnel.
    /// &lt;/summary&gt;
    public static class G06TelemetryWire
    {
        public static TelemetryService CreateService(IAnalyticsReporter reporter = null)
        {
            var telemetry = new TelemetryService();
            telemetry.AddSink(new DebugLogSink());
            if (reporter == null &amp;&amp; ServiceLocator.TryGet&lt;IAnalyticsReporter&gt;(out var fromLocator))
                reporter = fromLocator;
            if (reporter != null) telemetry.AddSink(new WxAnalyticsSink(reporter));
            return telemetry;
        }
    }
}
// G06WxAnalyticsReporter.cs
#if SY_WECHAT
using System.Runtime.InteropServices;
using Sy.Telemetry;
```

```
namespace Sy.G06.Boot
{
    /// &lt;summary&gt;
    /// WeChat analytics reporter (task S2-T002, G03 paradigm): bridges the telemetry
    /// WxAnalyticsSink onto the platform JS entry SyReportAnalytics declared in
    /// SyWxBridge.jslib. Compiled only for real WeChat builds (no UNITY_EDITOR);
    /// mock/editor builds compile this out and the debug sink stays the only one.
    /// The duplicate extern declaration binds to the same jslib function the
    /// Sy.Platform Wx bridge uses.
    /// &lt;/summary&gt;
    public sealed class G06WxAnalyticsReporter : IAnalyticsReporter
    {
        [DllImport("__Internal", EntryPoint = "SyReportAnalytics")]
        static extern void ReportBridge(string evtId, string fieldsJson);
        public void Report(string evtId, string fieldsJson)
            =&gt; ReportBridge(evtId ?? "", fieldsJson ?? "{}");
    }
}
#endif
// G06AlbumPresenter.cs
using Sy.G06.Boot;
namespace Sy.G06
{
    /// &lt;summary&gt;Album row state for the view (task S2-T005 slice B).&lt;/summary&gt;
    public enum G06AlbumRow
    {
        Locked,    // not owned yet: price + buy seat + direct-pick seat
        Owned,     // bought/granted, not currently equipped: equip seat
        Equipped   // the live arrow feather: badge only, no seat
    }
    /// &lt;summary&gt;Direct-pick precheck verdict (the ad never fires on a dead row).&lt;/summary&gt;
    public enum G06DirectPrecheck
    {
        Launch,    // unowned + budget left: the view may fire the rewarded seat
        Blocked,   // today's direct budget is drained: no dead ad buttons (honest no-show)
        Owned      // stale locked row: never burn the cap on nothing
    }
    /// &lt;summary&gt;
    /// Pure album view-model (task S2-T005 slice B, BoardPresenter paradigm: no
    /// MonoBehaviour, no UGUI - EditMode tests pin every flow). Owns the three
    /// panel flows end to end: the buy/equip rows derive from the collection
    /// service (single wallet authority), and the direct-pick seat runs the
    /// G03 honest-offer law - CanOffer pre-check on the SAME funnel the accept
    /// consumes from, offer/blocked/granted/ad_fail telemetry on every state
    /// transition, and an owned row never reaches the ad. The view renders rows
    /// and forwards taps; it never re-decides a verdict.
    /// &lt;/summary&gt;
    public sealed class G06AlbumPresenter
    {
        readonly G06CollectionService _collection;
```

```
readonly G06AdFunnel _funnel;   // null on a degraded boot: the direct seat stays hidden
public G06AlbumPresenter(G06CollectionService collection, G06AdFunnel funnel)
{
    _collection = collection;
    _funnel = funnel;
}
// --- read faces for the panel + HUD ---
public int Wallet =&gt; _collection.ShardBalance;
public int OwnedCount =&gt; _collection.OwnedCount;
public int TotalSkins =&gt; _collection.TotalSkins;
public float CollectionProgress =&gt; _collection.CollectionProgress;
/// &lt;summary&gt;Catalog row order for the panel render (read face).&lt;/summary&gt;
public string[] SkinIds()
{
    return _collection.SkinIdList();
}
/// &lt;summary&gt;Render-time visibility of the direct seat: locked rows only,
/// and only while today's direct budget can still fire (no dead ad
/// buttons). Pure query - no telemetry, safe to call per frame.&lt;/summary&gt;
public bool DirectSeatVisible(string skinId)
{
    if (_collection.IsOwned(skinId)) return false;
    return _funnel != null &amp;&amp; _funnel.CanOffer(G06AdPoints.SkinPick);
}
public G06AlbumRow RowStateOf(string skinId)
{
    if (!_collection.IsOwned(skinId)) return G06AlbumRow.Locked;
    return _collection.EquippedSkin == skinId ? G06AlbumRow.Equipped : G06AlbumRow.Owned;
}
public int PriceOf(string skinId)
{
    var skin = _collection.SkinDataOf(skinId);
    return skin == null ? -1 : skin.shardCost;
}
public string NameKeyOf(string skinId)
{
    var skin = _collection.SkinDataOf(skinId);
    return skin == null ? string.Empty : skin.nameKey;
}
/// &lt;summary&gt;Raw catalog hex (with '#') for the row swatch; null when unknown.&lt;/summary&gt;
public string ColorHexOf(string skinId)
{
    var skin = _collection.SkinDataOf(skinId);
    return skin == null ? null : skin.colorHex;
}
/// &lt;summary&gt;Equipped feather hex for the tower tint face; null = stock ink.&lt;/summary&gt;
public string EquippedSkinHex()
{
    var id = _collection.EquippedSkin;
    if (string.IsNullOrEmpty(id)) return null;
```

```
            return ColorHexOf(id);
        }
        // --- tap flows (each resolves through the wallet authority) ---
        /// &lt;summary&gt;Buy seat: passthrough so the rejection law (zero mutation zero
        /// persist) stays enforced inside the service, where the tests pin it.&lt;/summary&gt;
        public G06BuyResult PressBuy(string skinId)
        {
            return _collection.BuySkin(skinId);
        }
        /// &lt;summary&gt;Equip seat: owned ids only, passthrough to the authority.&lt;/summary&gt;
        public G06EquipResult PressEquip(string skinId)
        {
            return _collection.EquipSkin(skinId);
        }
        /// &lt;summary&gt;Direct-pick precheck + honest offer telemetry. Launch already
        /// reports offer; Blocked reports blocked (the audit face of a no-show);
        /// Owned stays silent (a stale row is a render bug, not an ad event).&lt;/summary&gt;
        public G06DirectPrecheck RequestDirect(string skinId)
        {
            if (_collection.IsOwned(skinId)) return G06DirectPrecheck.Owned;
            if (_funnel == null || !_funnel.CanOffer(G06AdPoints.SkinPick))
            {
                _funnel?.ReportBlocked(G06AdPoints.SkinPick, "skin_pick");
                return G06DirectPrecheck.Blocked;
            }
            _funnel.ReportOffer(G06AdPoints.SkinPick, "skin_pick");
            return G06DirectPrecheck.Launch;
        }
        /// &lt;summary&gt;Ad completed: bank the grant. Ok reports granted; a stale
        /// AlreadyOwned reject reports ad_fail with the stale marker - the seat
        /// was consumed but nothing landed, and the funnel ledger must say so.&lt;/summary&gt;
        public G06GrantResult OnDirectGranted(string skinId)
        {
            var result = _collection.GrantSkin(skinId);
            if (result == G06GrantResult.Ok) _funnel?.ReportGranted(G06AdPoints.SkinPick, "skin_pick");
            else if (result == G06GrantResult.AlreadyOwned)
                _funnel?.ReportAdFail(G06AdPoints.SkinPick, "skin_pick_stale");
            return result;
        }
        /// &lt;summary&gt;Ad failed or was cancelled: the wallet never moved, the ledger says so.&lt;/summary&gt;
        public void OnDirectFailed()
        {
            _funnel?.ReportAdFail(G06AdPoints.SkinPick, "skin_pick");
        }
        /// &lt;summary&gt;Income seat: perfect drops bank through the wallet authority.&lt;/summary&gt;
        public bool Deposit(int amount)
        {
            return _collection.DepositShards(amount);
        }
    }
```

```
}
// G06ArtWiring.cs
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.G06
{
    /// &lt;summary&gt;
    /// Art wiring base for the four registered S2.5 work-order slots (docs/06
    /// section 7, zero-asset-first per X073 prefix rule): resolves each slot
    /// through the shared ArtCatalog (X027) and falls back to the procedural
    /// placeholder whenever the sprite is not on disk yet. Consumers MUST treat
    /// a null sprite as "keep procedural rendering" so headless checks stay
    /// green on any project state (art absent, catalog absent, name typo).
    /// Slot -&gt; folder map (ArtSetup category caps apply by folder):
    ///   g06_range_base        -&gt; Assets/_Game/Art/Scene   (1024 cap)
    ///   g06_arrow_hero        -&gt; Assets/_Game/Art/Units   (512 cap)
    ///   g06_fletch_skin_sheet -&gt; Assets/_Game/Art/Units   (512 cap)
    ///   g06_share_card_face   -&gt; Assets/_Game/Art/Share   (1024 cap)
    /// Formal art arrives strictly via the TJGenerators leg (U020); drafts and
    /// procedural placeholders never ship.
    /// &lt;/summary&gt;
    public static class G06ArtWiring
    {
        public const string RangeBase = "g06_range_base";
        public const string ArrowHero = "g06_arrow_hero";
        public const string FletchSkinSheet = "g06_fletch_skin_sheet";
        public const string ShareCardFace = "g06_share_card_face";
        public static readonly string[] Slots =
        {
            RangeBase, ArrowHero, FletchSkinSheet, ShareCardFace,
        };
        /// &lt;summary&gt;Resolved sprite for a slot, or null when art has not landed
        /// (catalog absent or key missing). Never throws.&lt;/summary&gt;
        public static Sprite SpriteOrNull(string slot)
        {
            if (string.IsNullOrEmpty(slot)) return null;
            return ArtRuntime.Get(slot);
        }
        /// &lt;summary&gt;True when the procedural placeholder must keep rendering for
        /// this slot (no sprite on disk yet).&lt;/summary&gt;
        public static bool FallbackActive(string slot)
        {
            return SpriteOrNull(slot) == null;
        }
        /// &lt;summary&gt;Slot inventory sanity for checks: exactly the four
        /// registered S2.5 work orders, non-empty and unique.&lt;/summary&gt;
        public static bool SlotsHealthy()
        {
            if (Slots == null || Slots.Length != 4) return false;
```

```
            var seen = new HashSet&lt;string&gt;(System.StringComparer.Ordinal);
            for (int i = 0; i &lt; Slots.Length; i++)
            {
                string s = Slots[i];
                if (string.IsNullOrEmpty(s)) return false;
                if (!seen.Add(s)) return false;
            }
            return true;
        }
    }
}
// G06BeatFeel.cs
using System;
namespace Sy.G06
{
    /// &lt;summary&gt;
    /// Pure beat-feel solver for the S1-T004 presentation beats (slice A: stack
    /// landing bounce + first-wow boast banner). Zero UnityEngine types: EditMode
    /// tests pin the curves (G06BeatFeelTests); the UGUI host only consumes. Every
    /// beat keeps a distinct signature (duration + peak + shape) so the four
    /// performances stay perceivable and distinguishable in the event stream
    /// (task acceptance: each beat perceivable, event stream distinguishable).
    /// Sim semantics stay frozen (E4 anchor) - this layer only shapes feel.
    /// &lt;/summary&gt;
    public static class G06BeatFeel
    {
        // -- stack landing bounce (growth beat: the fresh arrow drops in and settles) --
        public const float StackBounceSec = 0.30f;  // total beat duration (snappy, joy-forward)
        public const float StackDropIn = 120f;      // board units above rest at k=0 (visible entry)
        public const float StackRebound = 26f;      // rebound overshoot after impact (visible life)
        public const float StackSquash = 0.34f;    // scale-y squash depth at impact (1 - squash)
        public const float StackStretch = 1.22f;    // scale stretch at rebound peak
        public const float StackFallStretch = 0.08f; // slight stretch while the arrow falls
        // phase splits of the bounce (normalized beat time 0..1)
        public const float StackImpactK = 0.45f;
        public const float StackPeakK = 0.625f;
        public const float StackSettleK = 0.80f;
        // -- first-wow boast banner (height-3 milestone: share/boast seed) --
        public const float WowBannerSec = 1.6f;     // banner lifetime (readable boast window)
        public const float WowPunch = 1.18f;         // reveal punch scale (visible pop)
        public const float WowPreScale = 0.92f;     // pre-reveal scale (grows into the punch)
        public const float WowRise = 34f;           // board units rise over the lifetime
        public const float WowAttackK = 0.12f;      // punch-in fraction
        public const float WowSettleK = 0.45f;      // back-to-rest fraction
        static float Clamp01(float k)
        {
            if (k &lt; 0f) return 0f;
            if (k &gt; 1f) return 1f;
            return k;
        }
```

```
/// &lt;summary&gt;
/// Vertical board-space offset of the freshly stacked arrow during the landing
/// bounce, k = normalized beat time 0..1. Shape: gravity ease-in drop from
/// StackDropIn, impact at StackImpactK, rebound arc peaking StackRebound,
/// settled at exactly 0 from StackSettleK on.
/// &lt;/summary&gt;
public static float StackBounceOffset(float k)
{
    k = Clamp01(k);
    if (k &lt; StackImpactK)
    {
        float t = k / StackImpactK;
        return StackDropIn * (1f - t * t);   // accelerating fall
    }
    if (k &lt; StackSettleK)
    {
        float t = (k - StackImpactK) / (StackSettleK - StackImpactK);
        return StackRebound * (float)Math.Sin(Math.PI * t);
    }
    return 0f;
}
/// &lt;summary&gt;
/// Squash-and-stretch scales of the stacked arrow during the bounce:
/// slight stretch while falling, impact squash snap, rebound stretch,
/// rest at exactly (1, 1) at k=1. sx conserves the read (squash widens,
/// stretch narrows) so the arrow keeps its volume on screen.
/// &lt;/summary&gt;
public static void StackBounceScale(float k, out float sx, out float sy)
{
    k = Clamp01(k);
    if (k &lt; StackImpactK)
    {
        float t = k / StackImpactK;
        sy = 1f + StackFallStretch * (1f - t);
    }
    else if (k &lt; StackPeakK)
    {
        // impact snap: squash depth at the frame of contact, rising to stretch
        float t = (k - StackImpactK) / (StackPeakK - StackImpactK);
        float squashBottom = 1f - StackSquash;
        sy = squashBottom + (StackStretch - squashBottom) * t;
    }
    else
    {
        float t = (k - StackPeakK) / (1f - StackPeakK);
        sy = StackStretch + (1f - StackStretch) * t;
    }
    sx = 2f - sy;
}
/// &lt;summary&gt;
```

```
/// Uniform scale of the wow banner over its lifetime: pre-reveal small,
/// fast attack into the WowPunch pop, ease back to rest at WowSettleK,
/// exactly 1 from there on.
/// &lt;/summary&gt;
public static float WowBannerScale(float k)
{
    k = Clamp01(k);
    if (k &lt; WowAttackK)
    {
        float t = k / WowAttackK;
        return WowPreScale + (WowPunch - WowPreScale) * t;
    }
    if (k &lt; WowSettleK)
    {
        float t = (k - WowAttackK) / (WowSettleK - WowAttackK);
        return WowPunch + (1f - WowPunch) * t;
    }
    return 1f;
}
/// &lt;summary&gt;Vertical rise of the wow banner over its lifetime (linear drift).&lt;/summary&gt;
public static float WowBannerRise(float k)
{
    return WowRise * Clamp01(k);
}
// -- collapse arrow-rain slow-mo (slice B: the wow point - the tower bursts,
//    the whole screen dilates into slow motion and its arrows rain down) --
public const float CollapseSec = 1.15f;        // beat time (distinct: 0.30 / 1.6 / 1.15)
public const float CollapseTimeFloor = 0.22f;  // deep slow-mo dip (the freeze read)
public const float CollapseDipK = 0.10f;       // 1 -&gt; floor within this fraction (impact snap)
public const float CollapseHoldK = 0.68f;      // floor held until here, then releases
public const int CollapseRainCount = 12;       // rain density (full-screen wow face)
public const float CollapseRainXSpread = 470f;  // half-width of the rain band (canvas 1080)
public const float CollapseRainTopY = 1120f;    // spawn above the visible top (y +960)
public const float CollapseRainBottomY = -1080f; // rest below the visible bottom (y -960)
public const float CollapseRainMaxDelayK = 0.15f; // stagger ceiling (fraction of the beat)
/// &lt;summary&gt;
/// World time scale during the collapse beat, k = normalized beat time 0..1.
/// Hard snap from real time into the freeze floor by CollapseDipK, held deep,
/// eased back to exactly 1 at k=1 - the app is never left dilated.
/// &lt;/summary&gt;
public static float CollapseTimeScale(float k)
{
    k = Clamp01(k);
    if (k &lt; CollapseDipK)
    {
        float t = k / CollapseDipK;
        return 1f + (CollapseTimeFloor - 1f) * t;
    }
    if (k &lt; CollapseHoldK) return CollapseTimeFloor;
    float u = (k - CollapseHoldK) / (1f - CollapseHoldK);
```

```
    return CollapseTimeFloor + (1f - CollapseTimeFloor) * u;
}
/// &lt;summary&gt;Deterministic rain slot: golden-stride spread, all distinct, in band.&lt;/summary&gt;
public static float CollapseRainX(int i)
{
    return ((i * 173) % 941) - CollapseRainXSpread;
}
/// &lt;summary&gt;Staggered entry so the rain reads as a burst, not a curtain drop.&lt;/summary&gt;
public static float CollapseRainDelay(int i)
{
    return CollapseRainMaxDelayK * ((i * 3) % 4) / 3f;
}
/// &lt;summary&gt;
/// Canvas-space y of rain arrow i at beat time k: spawns above the visible
/// top at staggered heights, falls with quadratic acceleration (gravity
/// read) to below the visible bottom by k=1.
/// &lt;/summary&gt;
public static float CollapseRainY(int i, float k)
{
    k = Clamp01(k);
    float p = Clamp01((k - CollapseRainDelay(i)) / (1f - CollapseRainMaxDelayK));
    float start = CollapseRainTopY + ((i * 61) % 5) * 80f;
    return start + (CollapseRainBottomY - start) * p * p;
}
/// &lt;summary&gt;Fixed scattered tilt per arrow (deterministic, no spin integration).&lt;/summary&gt;
public static float CollapseRainTilt(int i)
{
    return ((i * 47) % 90) - 45f;
}
/// &lt;summary&gt;Rain alpha: full read until the tail, then a linear fade to zero.&lt;/summary&gt;
public static float CollapseRainAlpha(float k)
{
    k = Clamp01(k);
    if (k &lt; 0.82f) return 1f;
    return 1f - (k - 0.82f) / 0.18f;
}
// -- miss skew quip easter egg (slice C: the "so close" humor beat, U023) --
// A release a hair under the good band reads as "pinned askew on the tower
// edge": the arrow leans out, slides off, and a quip bubble roasts the
// near miss. Distinct beat signatures (0.30 / 1.6 / 1.15 / 1.05) keep the
// four performances distinguishable in the event stream.
public const float MissNearBand = 0.08f;       // near-miss band under GoodMin (0.52..0.60)
public const float MissWildThreshold = 0.08f;  // wild whiff face (barely drew the bow)
public const float MissSkewMaxDeg = 38f;       // steepest lean at the band edge
public const int NearQuipCount = 4;            // rotating near-miss quip family size
public const float MissSlideUnits = 150f;      // sideways slide of the askew arrow
public enum MissKind { NearEdge, EarlySlip, WildWhiff }
/// &lt;summary&gt;
/// Miss flavor from the release power. Fed ONLY with miss verdicts
/// (power &amp;lt; ArrowRushSim.GoodMin) - the view classifies after the sim has
```

```
/// judged the beat, so this never re-decides a verdict (E4 tree frozen).
/// &lt;/summary&gt;
public static MissKind MissKindOf(float power)
{
    if (power &lt; 0f) power = 0f;
    if (power &gt;= ArrowRushSim.GoodMin - MissNearBand &amp;&amp; power &lt; ArrowRushSim.GoodMin)
        return MissKind.NearEdge;
    if (power &lt; MissWildThreshold) return MissKind.WildWhiff;
    return MissKind.EarlySlip;
}
/// &lt;summary&gt;
/// Lean angle of the dropped arrow: the closer the release sat to the good
/// band, the steeper the pinned-askew read; wild whiffs stay near flat
/// (the arrow never reached the tower). Monotonic in power, 0..SkewMax.
/// &lt;/summary&gt;
public static float MissSkewDeg(float power)
{
    if (power &lt;= 0f) return 0f;
    if (power &gt; ArrowRushSim.GoodMin) power = ArrowRushSim.GoodMin;
    float u = power / ArrowRushSim.GoodMin;
    return MissSkewMaxDeg * u * u;
}
/// &lt;summary&gt;Deterministic quip rotation by miss ordinal (no Random in beats).&lt;/summary&gt;
public static int QuipIndex(int missOrdinal, int quipCount)
{
    if (quipCount &lt;= 0) return 0;
    int i = missOrdinal % quipCount;
    return i &lt; 0 ? i + quipCount : i;
}
// quip bubble panel beat (the easter-egg face for near-edge misses)
public const float QuipPanelSec = 1.05f;   // lifetime (distinct from 0.30 / 1.6 / 1.15)
public const float QuipPunch = 1.14f;      // reveal pop
public const float QuipPreScale = 0.86f;   // pre-reveal scale
public const float QuipAttackK = 0.14f;    // punch-in fraction
public const float QuipSettleK = 0.50f;    // back-to-rest fraction
/// &lt;summary&gt;
/// Bubble scale over its lifetime: pre-reveal grow, pop punch, eased back
/// to exactly 1 from QuipSettleK on.
/// &lt;/summary&gt;
public static float QuipPanelScale(float k)
{
    k = Clamp01(k);
    if (k &lt; QuipAttackK)
    {
        float t = k / QuipAttackK;
        return QuipPreScale + (QuipPunch - QuipPreScale) * t;
    }
    if (k &lt; QuipSettleK)
    {
        float t = (k - QuipAttackK) / (QuipSettleK - QuipAttackK);
```

```
                return QuipPunch + (1f - QuipPunch) * t;
            }
            return 1f;
        }
        /// &lt;summary&gt;Sideways slide of the askew arrow over the beat (linear drift out).&lt;/summary&gt;
        public static float MissSlideOffset(float k)
        {
            return MissSlideUnits * Clamp01(k);
        }
    }
}
// G06BoardPresenter.cs
using System;
using System.Collections.Generic;
namespace Sy.G06
{
    /// &lt;summary&gt;One presentation beat derived from an ArrowRushSim event (task S1-T001).&lt;/summary&gt;
    public sealed class G06ViewCommand
    {
        public string Kind;       // caption | guide | shot_released | arrow_split | arrow_landed | stack | split_stack
        public string Beat;       // Perfect | Good | Miss | None (parsed from beat= when present)
        public float Power;       // release charge 0..1 (shot_released)
        public int Shot;          // shot ordinal (shot_released / arrow_landed)
        public int Height;        // tower height (stack / split_stack / first_wow)
        public float Stability;   // post-miss stability (arrow_dropped)
        public int Shards;        // shard total (shard)
        public int Collapse;      // collapse ordinal (collapse)
        public int Splits;        // fork arrow count (arrow_split, S2-T003 slice B)
        public float Fraction;    // fork height fraction of flight (arrow_split)
        public int ArrowIndex;    // fork arrow ordinal 1..Splits (split_stack)
        public int WindDir;       // wind direction on wind_shift: +1 tail / -1 head (S2-T004 slice B)
        public float TimeSec;      // sim-seconds timestamp
        public string Detail;     // original sim detail (parity trace)
    }
    /// &lt;summary&gt;
    /// Pure view-model for ArrowRush (task S1-T001): owns the ArrowRushSim, converts
    /// hold/release input into sim calls, and drains sim events into typed view
    /// commands 1:1. No MonoBehaviour and no UGUI types: EditMode tests pin the command
    /// stream against sim events, and the E4 13-assert tree stays the permanent anchor
    /// (sim semantics are frozen - the presenter only maps, never re-decides).
    /// Input contract for the view (task S1-T003): BeginHold / ReleaseShot / Advance;
    /// charge arc reads CurrentPower, stability slot reads StabilityFraction.
    /// &lt;/summary&gt;
    public sealed class G06BoardPresenter
    {
        readonly ArrowRushSim _sim;
        readonly List&lt;G06ViewCommand&gt; _drained = new List&lt;G06ViewCommand&gt;();
        int _drainIndex;
        public G06BoardPresenter()
        {
```

```
    _sim = new ArrowRushSim();
}
public ArrowRushSim Sim =&gt; _sim;
// -- read faces for the view (no sim mutation) --
public ArrowRushSim.Phase Phase =&gt; _sim.CurrentPhase;
/// &lt;summary&gt;Tower height in arrows (render face for the stacked tower).&lt;/summary&gt;
public int StackHeight =&gt; _sim.StackHeight;
/// &lt;summary&gt;Shard collectible count (hybrid seat 2 HUD).&lt;/summary&gt;
public int ShardCount =&gt; _sim.ShardCount;
/// &lt;summary&gt;Live charge power 0..1 while holding (arc bar source, S1-T003 view face).&lt;/summary&gt;
public float CurrentPower =&gt; _sim.Charge;
/// &lt;summary&gt;Stability slot fraction 0..1 of the [-4,+4] band for the bar UI.&lt;/summary&gt;
public float StabilityFraction
{
    get
    {
        const float max = ArrowRushSim.StabilityMax;
        float v = _sim.Stability / max;
        return Math.Min(1f, Math.Max(0f, v));
    }
}
// -- input contract (sim mutation, then drain) --
/// &lt;summary&gt;Opening hook: caption lands at t=0 (red line 11).&lt;/summary&gt;
public G06ViewCommand[] Open()
{
    _sim.Open();
    return Drain();
}
/// &lt;summary&gt;First-time guide prompt (once per run).&lt;/summary&gt;
public G06ViewCommand[] MaybeShowGuide()
{
    _sim.MaybeShowGuide();
    return Drain();
}
/// &lt;summary&gt;
/// Opening hook (task S1-T005): runs the scripted E4 timeline through the
/// frozen sim and drains the whole beat stream for the view replay queue.
/// The command stream is the production face of the ten-second hook -
/// EditMode tests pin it against the E4 evidence numbers.
/// &lt;/summary&gt;
public G06ViewCommand[] RunOpening()
{
    G06HookFlow.Run(_sim);
    return Drain();
}
/// &lt;summary&gt;Press: start charging from zero.&lt;/summary&gt;
public void BeginHold()
{
    _sim.BeginHold();
}
```

```
/// &lt;summary&gt;Release the shot; returns the beat and drains the beat stream.&lt;/summary&gt;
public ArrowRushSim.Beat ReleaseShot(out G06ViewCommand[] beats)
{
    var beat = _sim.ReleaseShot();
    beats = Drain();
    return beat;
}
/// &lt;summary&gt;Clock pass-through (charge oscillation + cadence); drains whatever fired.&lt;/summary&gt;
public G06ViewCommand[] Advance(float dtSec)
{
    _sim.AdvanceClock(dtSec);
    return Drain();
}
/// &lt;summary&gt;Revive consumer (hybrid seat 1): restore play, drain the accepted beat.&lt;/summary&gt;
public G06ViewCommand[] AcceptRevive()
{
    _sim.AcceptRevive();
    return Drain();
}
/// &lt;summary&gt;Art stem convention for the art pipeline: g06_&amp;lt;id&amp;gt;.&lt;/summary&gt;
public static string ArtStem(string id)
{
    return string.IsNullOrEmpty(id) ? string.Empty : "g06_" + id;
}
// -- wind hourglass read faces (task S2-T004 slice B, HUD source) --
/// &lt;summary&gt;Wind layer live in this run (production opts in; frozen test paths never do).&lt;/summary&gt;
public bool WindEnabled =&gt; _sim.WindEnabled;
/// &lt;summary&gt;Live wind direction: +1 tail / -1 head (chevron source).&lt;/summary&gt;
public int WindDirection =&gt; _sim.WindDirection;
/// &lt;summary&gt;Seconds until the next hourglass flip (sand drain source).&lt;/summary&gt;
public float WindPhaseRemaining =&gt; _sim.WindPhaseRemaining;
/// &lt;summary&gt;The post head-&gt;tail catch window is live now (halo affordance source).&lt;/summary&gt;
public bool WindCatchOpen =&gt; _sim.WindCatchOpen;
/// &lt;summary&gt;Production opt-in for the wind hourglass (additive layer, default OFF).&lt;/summary&gt;
public void EnableWind()
{
    _sim.EnableWind();
}
// -- drain core (1:1 mapping, zero re-deciding) --
G06ViewCommand[] Drain()
{
    _drained.Clear();
    var events = _sim.Events;
    for (int i = _drainIndex; i &lt; events.Count; i++)
    {
        var e = events[i];
        var cmd = new G06ViewCommand
        {
            Kind = MapKind(e.Kind),
            Beat = ParseBeat(e.Detail),
```

```
                Power = ParseFloat(e.Detail, "power="),
                Shot = ParseInt(e.Detail, "shot="),
                Height = ParseInt(e.Detail, "height="),
                Stability = ParseFloat(e.Detail, "stability="),
                Shards = ParseInt(e.Detail, "shards="),
                Collapse = ParseInt(e.Detail, "collapse="),
                Splits = ParseInt(e.Detail, "splits="),
                Fraction = ParseFloat(e.Detail, "fraction="),
                ArrowIndex = ParseInt(e.Detail, "arrow="),
                WindDir = ParseWindDir(e.Detail),
                TimeSec = e.SimSeconds,
                Detail = e.Detail,
            };
            _drained.Add(cmd);
        }
        _drainIndex = events.Count;
        return _drained.ToArray();
    }
    static string MapKind(string kind)
    {
        switch (kind)
        {
            case "caption": return "caption";
            case "guide_prompt": return "guide";
            case "shot_released": return "shot_released";
            case "arrow_split": return "arrow_split";
            case "arrow_landed": return "arrow_landed";
            case "stack_signal": return "stack";
            case "split_stack": return "split_stack";
            case "arrow_dropped": return "arrow_dropped";
            case "shard_drop": return "shard";
            case "tower_collapse": return "collapse";
            case "revive_offer": return "revive_offer";
            case "revive_accepted": return "revive_accepted";
            case "first_wow": return "first_wow";
            case "wind_shift": return "wind_shift";
            case "wind_catch": return "wind_catch";
            default: return "resync";   // unknown beat: view resyncs from sim state
        }
    }
    /// &lt;summary&gt;Wind direction token of a wind_shift detail: dir=tail -&gt; +1, dir=head -&gt; -1.&lt;/summar
    static int ParseWindDir(string detail)
    {
        if (string.IsNullOrEmpty(detail)) return 0;
        if (detail.IndexOf("dir=tail", StringComparison.Ordinal) &gt;= 0) return 1;
        if (detail.IndexOf("dir=head", StringComparison.Ordinal) &gt;= 0) return -1;
        return 0;
    }
    static string ParseBeat(string detail)
    {
```

```
    const string token = "beat=";
    if (string.IsNullOrEmpty(detail)) return string.Empty;
    int at = detail.IndexOf(token, StringComparison.Ordinal);
    if (at &lt; 0) return string.Empty;
    int pos = at + token.Length;
    int end = detail.IndexOf(' ', pos);
    return end &lt; 0 ? detail.Substring(pos) : detail.Substring(pos, end - pos);
}
static int ParseInt(string detail, string token)
{
    if (string.IsNullOrEmpty(detail)) return -1;
    int at = detail.IndexOf(token, StringComparison.Ordinal);
    if (at &lt; 0) return -1;
    int pos = at + token.Length;
    int value = 0;
    bool any = false;
    while (pos &lt; detail.Length &amp;&amp; detail[pos] &gt;= '0' &amp;&amp; detail[pos] &lt;= '9')
    {
        value = value * 10 + (detail[pos] - '0');
        pos++;
        any = true;
    }
    return any ? value : -1;
}
static float ParseFloat(string detail, string token)
{
    if (string.IsNullOrEmpty(detail)) return -1f;
    int at = detail.IndexOf(token, StringComparison.Ordinal);
    if (at &lt; 0) return -1f;
    int pos = at + token.Length;
    bool neg = false;
    if (pos &lt; detail.Length &amp;&amp; (detail[pos] == '-' || detail[pos] == '+'))
    {
        neg = detail[pos] == '-';
        pos++;
    }
    float value = 0f;
    bool any = false;
    while (pos &lt; detail.Length &amp;&amp; detail[pos] &gt;= '0' &amp;&amp; detail[pos] &lt;= '9')
    {
        value = value * 10f + (detail[pos] - '0');
        pos++;
        any = true;
    }
    if (any &amp;&amp; pos &lt; detail.Length &amp;&amp; detail[pos] == '.')
    {
        pos++;
        float scale = 0.1f;
        while (pos &lt; detail.Length &amp;&amp; detail[pos] &gt;= '0' &amp;&amp; detail[pos] &lt;= '9')
        {
```

```
                    value += (detail[pos] - '0') * scale;
                    scale *= 0.1f;
                    pos++;
                }
            }
            return any ? (neg ? -value : value) : -1f;
        }
    }
}
// G06ChargeArc.cs
using System;
namespace Sy.G06
{
    /// &lt;summary&gt;Tri-zone verdict of the charge meter (render mirror of ArrowRushSim.Classify).&lt;/summary&gt;
    public enum G06ChargeZone
    {
        Weak,
        Good,
        Perfect
    }
    /// &lt;summary&gt;
    /// Pure math solver for the S1-T003 charge arc view face (first slice, pure code):
    /// zone classification with parity to the frozen sim timing bands, arc geometry for
    /// the three-zone charge bar bending over the mid-rendered tower, the ink-teal
    /// trajectory signature (predicted arrow flight samples + linear fade), and the
    /// stability bar color lerp. No MonoBehaviour and no UGUI/UnityEngine types:
    /// EditMode tests pin the math (G06ChargeArcTests); the UGUI host (next slice)
    /// only consumes. Sim semantics stay frozen (E4 anchor) - this layer mirrors,
    /// never re-decides.
    /// &lt;/summary&gt;
    public static class G06ChargeArc
    {
        // -- zone palette (visual signature: target-ring gold x ink teal) --
        public const string WeakZoneHex = "22333B";     // ink teal base (weak sweep)
        public const string GoodZoneHex = "8C7326";     // dim gold (signature family, de-emphasized)
        public const string PerfectZoneHex = "F0A500";  // target-ring gold (sweet-spot band accent)
        public const string SafeStabilityHex = "22333B";   // stable tower: ink teal
        public const string DangerStabilityHex = "C0392B"; // stability drained: signal red (not a palette accent)
        // -- arc geometry (1080x1920 portrait board space; tower renders mid-screen) --
        public const float ArcCenterX = 0f;    // tower mid-render anchor = screen center x
        public const float ArcCenterY = 640f;  // tower base line in board space
        public const float ArcRadius = 380f;
        public const float ArcStartDeg = 150f; // upper-left rim
        public const float ArcEndDeg = 30f;    // upper-right rim (fill sweeps over the tower top)
        // -- trajectory signature (ink-teal fading arc preview during hold) --
        public const float LaunchAngleDeg = 62f;   // arrow launch angle above horizon
        public const float LaunchSpeedMin = 520f;  // board units/sec at power 0
        public const float LaunchSpeedMax = 1450f; // board units/sec at power 1
        public const float Gravity = 1600f;        // board units/sec^2
        /// &lt;summary&gt;
```

```
/// Verdict-parity zone of a power value. Mirrors ArrowRushSim.Classify exactly:
/// [PerfectMin, PerfectMax] is the sweet-spot band; above it the meter reads
/// Good (overcharge is a hit, not a perfect), below GoodMin it is a miss.
/// &lt;/summary&gt;
public static G06ChargeZone ZoneOf(float power)
{
    if (power &gt;= ArrowRushSim.PerfectMin &amp;&amp; power &lt;= ArrowRushSim.PerfectMax)
        return G06ChargeZone.Perfect;
    if (power &gt;= ArrowRushSim.GoodMin)
        return G06ChargeZone.Good;
    return G06ChargeZone.Weak;
}
/// &lt;summary&gt;
/// Meter t-range [start, end] a zone occupies on the 0..1 sweep. Bands tile
/// [0, PerfectMax] contiguously; the sliver (PerfectMax, 1] renders in the Good
/// family (verdict parity - the bar never paints a perfect color outside the band).
/// &lt;/summary&gt;
public static void ZoneBand(G06ChargeZone zone, out float start, out float end)
{
    switch (zone)
    {
        case G06ChargeZone.Perfect:
            start = ArrowRushSim.PerfectMin;
            end = ArrowRushSim.PerfectMax;
            return;
        case G06ChargeZone.Good:
            start = ArrowRushSim.GoodMin;
            end = ArrowRushSim.PerfectMin;
            return;
        default:
            start = 0f;
            end = ArrowRushSim.GoodMin;
            return;
    }
}
/// &lt;summary&gt;Maps a zone-local fraction 0..1 to the meter 0..1 range.&lt;/summary&gt;
public static float MeterT(G06ChargeZone zone, float localT)
{
    ZoneBand(zone, out float start, out float end);
    return start + (end - start) * Clamp01(localT);
}
/// &lt;summary&gt;Palette hex of a zone (view converts to its color type).&lt;/summary&gt;
public static string ZoneHex(G06ChargeZone zone)
{
    switch (zone)
    {
        case G06ChargeZone.Perfect: return PerfectZoneHex;
        case G06ChargeZone.Good: return GoodZoneHex;
        default: return WeakZoneHex;
    }
```

```
}
/// &lt;summary&gt;
/// Arc point at meter fraction t (0 = left rim, 1 = right rim). The bar bends
/// over the tower: the sweep passes the apex at t = 0.5.
/// &lt;/summary&gt;
public static void PointAt(float t, out float x, out float y)
{
    float deg = ArcStartDeg + (ArcEndDeg - ArcStartDeg) * Clamp01(t);
    double rad = deg * Math.PI / 180.0;
    x = ArcCenterX + ArcRadius * (float)Math.Cos(rad);
    y = ArcCenterY + ArcRadius * (float)Math.Sin(rad);
}
/// &lt;summary&gt;Launch speed for a power value (linear in the 0..1 sweep).&lt;/summary&gt;
public static float LaunchSpeedOf(float power)
{
    return LaunchSpeedMin + (LaunchSpeedMax - LaunchSpeedMin) * Clamp01(power);
}
/// &lt;summary&gt;Full flight duration until the arrow returns to launch height.&lt;/summary&gt;
public static float FlightDurationOf(float power)
{
    double rad = LaunchAngleDeg * Math.PI / 180.0;
    return 2f * LaunchSpeedOf(power) * (float)Math.Sin(rad) / Gravity;
}
/// &lt;summary&gt;Predicted trajectory sample at flight time (launch at origin, x forward, y up).&lt;/summary&g
public static void TrajectoryPoint(float power, float flightSec, out float x, out float y)
{
    double rad = LaunchAngleDeg * Math.PI / 180.0;
    float v = LaunchSpeedOf(power);
    float cos = (float)Math.Cos(rad);
    float sin = (float)Math.Sin(rad);
    x = v * cos * flightSec;
    y = v * sin * flightSec - 0.5f * Gravity * flightSec * flightSec;
}
/// &lt;summary&gt;Linear fade for the ink-teal trail: head 1 (opaque) to tail 0.&lt;/summary&gt;
public static float TrailAlphaAt(float t)
{
    return 1f - Clamp01(t);
}
/// &lt;summary&gt;
/// Stability bar color: fraction 1 = safe ink teal, 0 = danger red, linear in between.
/// &lt;/summary&gt;
public static string StabilityHex(float fraction)
{
    int[] safe = HexToRgb(SafeStabilityHex);
    int[] danger = HexToRgb(DangerStabilityHex);
    float k = Clamp01(fraction);
    int r = (int)Math.Round(danger[0] + (safe[0] - danger[0]) * k);
    int g = (int)Math.Round(danger[1] + (safe[1] - danger[1]) * k);
    int b = (int)Math.Round(danger[2] + (safe[2] - danger[2]) * k);
    return string.Format("{0:X2}{1:X2}{2:X2}", r, g, b);
```

```
        }
        static float Clamp01(float v)
        {
            if (v &lt; 0f) return 0f;
            if (v &gt; 1f) return 1f;
            return v;
        }
        static int[] HexToRgb(string hex)
        {
            int r = Convert.ToInt32(hex.Substring(0, 2), 16);
            int g = Convert.ToInt32(hex.Substring(2, 2), 16);
            int b = Convert.ToInt32(hex.Substring(4, 2), 16);
            return new int[] { r, g, b };
        }
    }
}
// G06CollectionService.cs
using System;
using Sy.Core;
namespace Sy.G06
{
    /// &lt;summary&gt;Outcome of a shard purchase (task S2-T005, G03 buy-flow paradigm).&lt;/summary&gt;
    public enum G06BuyResult
    {
        Ok,               // skin bought: shards deducted, ownership persisted
        UnknownSkin,      // id not in the catalog table
        AlreadyOwned,     // no double charge for a repeat purchase
        InsufficientShards // balance below shardCost, nothing mutated
    }
    /// &lt;summary&gt;Outcome of an equip switch (owned skins only; empty = stock arrow).&lt;/summary&gt;
    public enum G06EquipResult
    {
        Ok,
        UnknownSkin,
        NotOwned
    }
    /// &lt;summary&gt;Outcome of a direct ad-pick grant (G06AdPoints.SkinPick reward face).&lt;/summary&gt;
    public enum G06GrantResult
    {
        Ok,           // skin granted free of shards (rewarded seat, 1/day cap at the funnel layer)
        UnknownSkin,
        AlreadyOwned  // the panel must pre-filter owned ids so a daily cap is never burned on nothing
    }
    /// &lt;summary&gt;
    /// Feather-skin collection economy (task S2-T005 slice A). The single authority
    /// over the profile's shard balance, owned set and equipped skin: every mutation
    /// routes through here and persists immediately through G06SaveSystem (local
    /// authority only, red line 12). Rejections never mutate and never persist, so
    /// a failed purchase can never burn a balance the HUD did not show. The rewarded
    /// direct-pick grant is shard-free by design - the ad seat pays the price, the
```

```
/// wallet does not. The album panel (slice B) consumes the read faces; nothing
/// here touches UI.
/// &lt;/summary&gt;
public sealed class G06CollectionService
{
    readonly G06GameCatalog _catalog;
    readonly ISaveService _save;
    readonly G06Profile _profile;
    readonly string _slot;
    public G06CollectionService(G06GameCatalog catalog, ISaveService save, G06Profile profile,
        string slot = G06SaveSystem.Slot)
    {
        _catalog = catalog ?? throw new ArgumentNullException(nameof(catalog));
        _save = save;
        _profile = profile ?? throw new ArgumentNullException(nameof(profile));
        _slot = slot;
        if (_profile.ownedSkins == null) _profile.ownedSkins = new System.Collections.Generic.List&lt;string&gt;();
    }
    // --- read faces (HUD + album panel slice B) ---
    public int ShardBalance =&gt; _profile.shards;
    public string EquippedSkin =&gt; _profile.equippedSkin ?? "";
    public int OwnedCount =&gt; _profile.ownedSkins.Count;
    public int TotalSkins =&gt; _catalog.Skins.Length;
    /// &lt;summary&gt;Collection progress 0..1 for the HUD ring; 0 when the table is empty.&lt;/summary&gt;
    public float CollectionProgress
    {
        get
        {
            if (TotalSkins == 0) return 0f;
            return (float)OwnedCount / TotalSkins;
        }
    }
    public bool IsOwned(string skinId)
    {
        return _profile.ownedSkins.Contains(skinId);
    }
    public FeatherSkinData SkinDataOf(string skinId)
    {
        return _catalog.SkinById(skinId);
    }
    /// &lt;summary&gt;Catalog id order for the album row render (read face, slice B).&lt;/summary&gt;
    public string[] SkinIdList()
    {
        var ids = new string[_catalog.Skins.Length];
        for (int i = 0; i &lt; ids.Length; i++) ids[i] = _catalog.Skins[i].id;
        return ids;
    }
    // --- mutations (each persists on success, never on rejection) ---
    /// &lt;summary&gt;Shard purchase: deduct, own, persist. Rejections mutate nothing.&lt;/summary&gt;
    public G06BuyResult BuySkin(string skinId)
```

```
        {
            var skin = _catalog.SkinById(skinId);
            if (skin == null) return G06BuyResult.UnknownSkin;
            if (IsOwned(skinId)) return G06BuyResult.AlreadyOwned;
            if (_profile.shards &lt; skin.shardCost) return G06BuyResult.InsufficientShards;
            _profile.shards -= skin.shardCost;
            _profile.ownedSkins.Add(skinId);
            Persist();
            return G06BuyResult.Ok;
        }
        /// &lt;summary&gt;Equip switch: owned ids only, persists the equipped face.&lt;/summary&gt;
        public G06EquipResult EquipSkin(string skinId)
        {
            var skin = _catalog.SkinById(skinId);
            if (skin == null) return G06EquipResult.UnknownSkin;
            if (!IsOwned(skinId)) return G06EquipResult.NotOwned;
            _profile.equippedSkin = skinId;
            Persist();
            return G06EquipResult.Ok;
        }
        /// &lt;summary&gt;Direct-pick reward grant: owns the skin with zero shard deduction
        /// (the rewarded seat already paid). Already-owned rejects so the daily cap
        /// is never silently consumed by a stale panel row.&lt;/summary&gt;
        public G06GrantResult GrantSkin(string skinId)
        {
            var skin = _catalog.SkinById(skinId);
            if (skin == null) return G06GrantResult.UnknownSkin;
            if (IsOwned(skinId)) return G06GrantResult.AlreadyOwned;
            _profile.ownedSkins.Add(skinId);
            Persist();
            return G06GrantResult.Ok;
        }
        /// &lt;summary&gt;Shard income face (slice B): perfect drops and any future bonus
        /// seat bank through the single wallet authority too - the view never mutates
        /// the profile directly. Zero/negative amounts mutate nothing; a real amount
        /// persists immediately so a crash can never lose banked shards.&lt;/summary&gt;
        public bool DepositShards(int amount)
        {
            if (amount &lt;= 0) return false;
            _profile.shards += amount;
            Persist();
            return true;
        }
        void Persist()
        {
            G06SaveSystem.Persist(_save, _profile, _slot);
        }
    }
}
// G06GameCatalog.cs
```

```
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G06
{
    [Serializable]
    public sealed class FeatherSkinData
    {
        public string id;         // f01..f06 collection ladder ids
        public string nameKey;    // i18n key for the skin display name
        public string colorHex;   // feather tint for the equipped arrow trail
        public int shardCost;     // shard price (shard = technique currency)
    }
    [Serializable]
    public sealed class DifficultyStageData
    {
        public int stage;          // 0..N-1; stage 0 is the graybox parity anchor
        public float chargeSpeed;  // meter sweep speed (units/sec)
        public float perfectMin;   // perfect band lower edge [perfectMin, perfectMax]
        public float perfectMax;   // perfect band upper edge (overcharge above it misses)
        public float goodMin;      // good band floor
    }
    [Serializable]
    public sealed class CatalogEconomyData
    {
        public int shardPerPerfect;      // parity: the sim drops exactly 1 shard per perfect
        public int reviveSeatCapPerDay;  // g06_revive_restore seat cap (GDD sec 3)
        public int shardDoubleCapPerDay; // g06_shard_double seat cap
        public int arrowPickCapPerDay;   // g06_arrow_pick seat cap
        public int globalAdCapPerDay;    // shared global ad cap across all seats
    }
    [Serializable]
    public sealed class CatalogFile
    {
        public FeatherSkinData[] skins;
        public DifficultyStageData[] difficultyStages;
        public CatalogEconomyData economy;
    }
    /// &lt;summary&gt;
    /// Static game catalog for ArrowRush (task S1-T002). Data layer only, G03
    /// GameCatalog paradigm: json under Resources + JsonUtility parse + structural
    /// Validate shared by the EditMode tests and the future gate. Parity anchors:
    /// difficulty stage 0 must equal the frozen ArrowRushSim timing constants
    /// (the E4 graybox face), and the economy must keep shardPerPerfect at 1
    /// because the sim hardcodes one shard per perfect. The skin table feeds the
    /// S2 collection/dex slice; the stage seeds feed the future difficulty
    /// wiring (T003/T004 mechanics slices decide consumption).
    /// &lt;/summary&gt;
    public sealed class G06GameCatalog
```

```
using Sy.G06.Boot;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// Platform integration tests (task S2-T002, G03PlatformTests paradigm): G06 ad
    /// point category mapping with namespace isolation, category caps (rescue 4 /
    /// double 5 / direct 1) and the 12/day global cap through both a fresh policy and
    /// the boot-registered mock suite, the SharedPolicy pre-check/consume consistency
    /// gate (offer faces pre-check on the exact policy the suite consumes from - the
    /// over-cap offer never fires, the honest daily_cap verdict agrees), and the
    /// evt_ad funnel state coverage through a capture sink.
    /// &lt;/summary&gt;
    public class G06PlatformTests
    {
        const string FixedDay = "20990101"; // deterministic injected day key
        [Test]
        public void AdPointMapping_G06IdsMapToRescueDoubleDirect()
        {
            Assert.That(AdCategories.CategoryOfPoint(G06AdPoints.Revive), Is.EqualTo(AdCategories.Rescue));
            Assert.That(AdCategories.CategoryOfPoint(G06AdPoints.ShardDouble), Is.EqualTo(AdCategories.Double));
            Assert.That(AdCategories.CategoryOfPoint(G06AdPoints.SkinPick), Is.EqualTo(AdCategories.Direct));
            // Namespace isolation: P02/G02/G03 ids stay distinct from the G06 ones.
            Assert.That(AdCategories.CategoryOfPoint("clear_wave"), Is.EqualTo(AdCategories.Rescue));
            Assert.That(AdCategories.CategoryOfPoint("g02_rescue_retry"), Is.EqualTo(AdCategories.Rescue));
            Assert.That(AdCategories.CategoryOfPoint("g03_rescue_purge"), Is.EqualTo(AdCategories.Rescue));
            Assert.That(G06AdPoints.Revive, Is.Not.EqualTo("clear_wave"), "g06 id must never equal a P02 id");
            Assert.That(G06AdPoints.Revive, Is.Not.EqualTo("g03_rescue_purge"), "g06 id must never equal a G03 id");
            Assert.That(G06AdPoints.SkinPick, Is.Not.EqualTo("g03_dex_pick"), "g06 id must never equal a G03 id");
            // Warm-up point stays uncategorized (global cap only).
            Assert.That(AdCategories.CategoryOfPoint("boot_warmup"), Is.Null);
            // Design-table caps: rescue 4 / double 5 / direct 1 on top of the 12/day global.
            Assert.That(AdCapPolicy.CategoryCap(AdCategories.Rescue), Is.EqualTo(4));
            Assert.That(AdCapPolicy.CategoryCap(AdCategories.Double), Is.EqualTo(5));
            Assert.That(AdCapPolicy.CategoryCap(AdCategories.Direct), Is.EqualTo(1));
            Assert.That(AdCapPolicy.DefaultDailyCap, Is.EqualTo(12));
        }
        [Test]
        public void RescueCategory_ReviveSeatCapsAtFourPerDay()
        {
            var ads = new MockAdsService(new AdCapPolicy(), () =&gt; FixedDay);
            for (int i = 0; i &lt; 4; i++)
            {
                var r = ads.ShowRewarded(G06AdPoints.Revive).Result;
                Assert.That(r.Completed, Is.True, "revive view " + (i + 1) + " must pass (cap 4/day)");
            }
            var fifth = ads.ShowRewarded(G06AdPoints.Revive).Result;
            Assert.That(fifth.Completed, Is.False, "5th revive view must hit the category cap");
            Assert.That(fifth.Reason, Is.EqualTo("daily_cap"));
```

```
}
[Test]
public void DoubleCategory_ShardSettleCapsAtFivePerDay()
{
    var ads = new MockAdsService(new AdCapPolicy(), () =&gt; FixedDay);
    for (int i = 0; i &lt; 5; i++)
    {
        var r = ads.ShowRewarded(G06AdPoints.ShardDouble).Result;
        Assert.That(r.Completed, Is.True, "shard double view " + (i + 1) + " must pass (cap 5/day)");
    }
    var sixth = ads.ShowRewarded(G06AdPoints.ShardDouble).Result;
    Assert.That(sixth.Completed, Is.False, "6th shard double view must hit the category cap");
    Assert.That(sixth.Reason, Is.EqualTo("daily_cap"));
}
[Test]
public void DirectCategory_SkinPickCapsAtOnePerDay()
{
    var ads = new MockAdsService(new AdCapPolicy(), () =&gt; FixedDay);
    Assert.That(ads.ShowRewarded(G06AdPoints.SkinPick).Result.Completed, Is.True,
        "first skin pick view must pass (cap 1/day)");
    var second = ads.ShowRewarded(G06AdPoints.SkinPick).Result;
    Assert.That(second.Completed, Is.False, "2nd skin pick view must hit the category cap");
    Assert.That(second.Reason, Is.EqualTo("daily_cap"));
}
[Test]
public void GlobalCap_TwelveAcrossAllCategoriesPerDay()
{
    var ads = new MockAdsService(new AdCapPolicy(), () =&gt; FixedDay);
    int completed = 0;
    for (int i = 0; i &lt; 4; i++) if (ads.ShowRewarded(G06AdPoints.Revive).Result.Completed) completed++;
    for (int i = 0; i &lt; 5; i++) if (ads.ShowRewarded(G06AdPoints.ShardDouble).Result.Completed) completed++;
    Assert.That(completed, Is.EqualTo(9), "rescue 4 + double 5 must all pass");
    Assert.That(ads.ShowRewarded(G06AdPoints.SkinPick).Result.Completed, Is.True, "direct 1 must pass");
    // 10 category views + 2 uncategorized warm-ups = the 12/day global cap.
    for (int i = 0; i &lt; 2; i++)
        Assert.That(ads.ShowRewarded("boot_warmup").Result.Completed, Is.True);
    var over = ads.ShowRewarded("boot_warmup").Result;
    Assert.That(over.Completed, Is.False, "13th rewarded view must hit the global cap");
    Assert.That(over.Reason, Is.EqualTo("daily_cap"));
}
[Test]
public void BootChain_MockSuiteRegistersAndPrecheckMatchesConsumption()
{
    G06BootServices.RegisterAll(); // idempotent: first call in this domain registers
    Assert.That(G06BootServices.SharedPolicy, Is.Not.Null,
        "the shared policy face must exist for offer pre-checks");
    // Platform suite faces registered for every later offer face.
    Assert.That(ServiceLocator.TryGet&lt;IPlatformService&gt;(out _), Is.True, "suite platform service registere
    Assert.That(ServiceLocator.TryGet&lt;IStorage&gt;(out _), Is.True, "suite storage registered");
    // Pre-check and consumption share the SAME policy instance: while the
```

```
        // pre-check says budget, the suite's rewarded view completes; once the
        // category is drained both faces agree it is over (never a dead ad).
        string today = AdCapPolicy.TodayKey();   // the mock suite consumes on the UTC day key
        Assert.That(ServiceLocator.TryGet&lt;IAdsService&gt;(out var ads), Is.True);
        Assert.That(G06BootServices.SharedPolicy.HasCategoryBudget(today, AdCategories.Rescue), Is.True,
            "fresh domain: rescue budget must pre-check open");
        for (int i = 0; i &lt; 4; i++)
        {
            Assert.That(G06BootServices.SharedPolicy.HasCategoryBudget(today, AdCategories.Rescue), Is.True,
                "rescue budget must stay open for view " + (i + 1));
            Assert.That(ads.ShowRewarded(G06AdPoints.Revive).Result.Completed, Is.True,
                "suite revive view " + (i + 1) + " must pass while the pre-check says open");
        }
        Assert.That(G06BootServices.SharedPolicy.HasCategoryBudget(today, AdCategories.Rescue), Is.False,
            "drained rescue category must pre-check closed (over-cap offer never fires)");
        var capped = ads.ShowRewarded(G06AdPoints.Revive).Result;
        Assert.That(capped.Completed, Is.False,
            "suite ads must enforce the rescue 4/day category cap");
        Assert.That(capped.Reason, Is.EqualTo("daily_cap"));
        // Second register is a no-op (idempotent guard, no throw, policy untouched).
        var policyBefore = G06BootServices.SharedPolicy;
        G06BootServices.RegisterAll();
        Assert.That(G06BootServices.SharedPolicy, Is.SameAs(policyBefore),
            "idempotent re-register must never swap the shared policy instance");
    }
    [Test]
    public void AdFunnel_OverCapPrecheckRejectsOfferButNotOtherCategories()
    {
        var policy = new AdCapPolicy();
        var ads = new MockAdsService(policy, () =&gt; FixedDay);
        var funnel = new G06AdFunnel(null, ads, policy, () =&gt; FixedDay);
        Assert.That(funnel.CanOffer(G06AdPoints.Revive), Is.True, "fresh day: revive offer must be showable");
        for (int i = 0; i &lt; 4; i++) ads.ShowRewarded(G06AdPoints.Revive).Wait();
        Assert.That(funnel.CanOffer(G06AdPoints.Revive), Is.False,
            "drained rescue category must reject the offer face (honest no-show)");
        Assert.That(funnel.CanOffer(G06AdPoints.ShardDouble), Is.True,
            "rescue drain must not touch the double category budget");
        Assert.That(funnel.CanOffer("boot_warmup"), Is.True,
            "uncategorized points always pass the pre-check (global cap only)");
    }
    [Test]
    public void AdFunnel_EvtAdStatesReachTelemetrySink()
    {
        var telemetry = new TelemetryService();
        var sink = new CaptureSink();
        telemetry.AddSink(sink);
        var policy = new AdCapPolicy();
        var ads = new MockAdsService(policy, () =&gt; FixedDay);
        var funnel = new G06AdFunnel(telemetry, ads, policy, () =&gt; FixedDay);
        funnel.ReportOffer(G06AdPoints.Revive, "revive");
```

```
        funnel.ReportBlocked(G06AdPoints.SkinPick, "skin_pick");
        var result = funnel.Accept(G06AdPoints.Revive, "revive").Result;
        Assert.That(result.Completed, Is.True, "fresh budget: the accepted revive view must complete");
        funnel.ReportGranted(G06AdPoints.Revive, "revive");
        funnel.ReportAdFail(G06AdPoints.ShardDouble, "shard_double");
        string[] expectedStates = { "offer", "blocked", "accept", "granted", "ad_fail" };
        Assert.That(sink.Ids.Count, Is.EqualTo(expectedStates.Length), "one evt_ad per funnel state");
        for (int i = 0; i &lt; expectedStates.Length; i++)
        {
            Assert.That(sink.Ids[i], Is.EqualTo(EventIds.Ad), "every funnel event is an evt_ad");
            Assert.That(sink.Fields[i]["state"], Is.EqualTo(expectedStates[i]),
                "evt_ad state " + i + " must read " + expectedStates[i]);
            Assert.That(sink.Fields[i]["point"], Is.Not.Null.And.Not.Empty, "evt_ad carries the point id");
            Assert.That(sink.Fields[i]["reason"], Is.Not.Null, "evt_ad carries the reason slug");
        }
        Assert.That(sink.Fields[2]["point"], Is.EqualTo(G06AdPoints.Revive));
        Assert.That(sink.Fields[2]["reason"], Is.EqualTo("revive"));
    }
    [Test]
    public void Telemetry_WxAnalyticsSinkJoinsWhenReporterAvailable()
    {
        var reporter = new CaptureReporter();
        var telemetry = G06TelemetryWire.CreateService(reporter);
        Assert.That(telemetry.SinkCount, Is.EqualTo(2), "debug sink + wx analytics sink");
        telemetry.Track("g06_platform_probe", new Dictionary&lt;string, string&gt; { { "phase", "boot" } });
        Assert.That(reporter.Ids, Does.Contain("g06_platform_probe"));
        Assert.That(reporter.Jsons[0], Does.Contain("boot"));
    }
    [Test]
    public void Telemetry_MockPathStaysDebugSinkOnly()
    {
        // Mock/editor domain registers no IAnalyticsReporter, so the standard
        // service carries exactly the debug sink. The real Wx reporter compiles
        // only under SY_WECHAT (proven by the WxCompileCheck shadow gate).
        var telemetry = G06TelemetryWire.CreateService();
        Assert.That(telemetry.SinkCount, Is.EqualTo(1), "no analytics reporter in the mock domain");
    }
    sealed class CaptureSink : TelemetryService.ISink
    {
        public readonly List&lt;string&gt; Ids = new List&lt;string&gt;();
        public readonly List&lt;Dictionary&lt;string, string&gt;&gt; Fields = new List&lt;Dictionary&lt;string, stri
        public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
        {
            Ids.Add(evtId);
            Fields.Add(fields);
        }
    }
    sealed class CaptureReporter : IAnalyticsReporter
    {
        public readonly List&lt;string&gt; Ids = new List&lt;string&gt;();
```

```
            public readonly List&lt;string&gt; Jsons = new List&lt;string&gt;();
            public void Report(string evtId, string fieldsJson)
            {
                Ids.Add(evtId);
                Jsons.Add(fieldsJson);
            }
        }
    }
}
// G06PresenterTests.cs
using System;
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// S1-T001 presenter mapping gate: the command stream must drain 1:1 against
    /// sim events with typed fields parsed from the parity detail strings.
    /// The E4 graybox tree (G06GrayboxE4Tests) stays the permanent semantic anchor;
    /// these tests pin the presenter layer only - sim semantics are never re-decided here.
    /// &lt;/summary&gt;
    public class G06PresenterTests
    {
        const float HoldToPerfect = 0.375f; // 0.375 * 2.4 = 0.90 power, inside perfect band
        const float HoldToGood = 0.28f;     // 0.28 * 2.4 = 0.672 power, good band (no shard, no wow)
        static G06BoardPresenter NewOpened()
        {
            var p = new G06BoardPresenter();
            p.Open();
            p.Advance(0.05f);
            p.MaybeShowGuide();
            return p;
        }
        static int CountOf(G06ViewCommand[] beats, string kind)
        {
            int n = 0;
            for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) n++;
            return n;
        }
        static G06ViewCommand FirstOf(G06ViewCommand[] beats, string kind)
        {
            for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) return beats[i];
            return null;
        }
        [Test]
        public void Map1_CaptionAndGuideDrainOneToOneInOrder()
        {
            var p = new G06BoardPresenter();
            var openBeats = p.Open();
            Assert.AreEqual(1, openBeats.Length, "open must drain exactly the caption beat");
```

```
        Assert.AreEqual("caption", openBeats[0].Kind);
        Assert.AreEqual(0f, openBeats[0].TimeSec, 1e-6f, "caption beat lands at t=0");
        Assert.AreEqual(p.Sim.Events[0].Detail, openBeats[0].Detail, "parity detail trace kept");
        var guideBeats = p.MaybeShowGuide();
        Assert.AreEqual(1, guideBeats.Length, "guide must drain exactly one beat");
        Assert.AreEqual("guide", guideBeats[0].Kind, "guide_prompt maps to guide");
        var again = p.MaybeShowGuide();
        Assert.AreEqual(0, again.Length, "second guide call drains nothing (once per run)");
    }
    [Test]
    public void Map2_PerfectShotCarriesTypedFields()
    {
        var p = NewOpened();
        p.BeginHold();
        var preCharge = p.CurrentPower;
        Assert.AreEqual(0f, preCharge, 1e-6f, "charge starts at zero on begin hold");
        p.Advance(HoldToPerfect);
        Assert.AreEqual(0.90f, p.CurrentPower, 0.01f, "charge must mirror the sim meter while holding");
        var beats = default(G06ViewCommand[]);
        var beat = p.ReleaseShot(out beats);
        Assert.AreEqual(ArrowRushSim.Beat.Perfect, beat, "0.90 power must classify perfect");
        Assert.AreEqual(1, CountOf(beats, "shot_released"), "one shot_released beat");
        Assert.AreEqual(1, CountOf(beats, "arrow_landed"), "one arrow_landed beat");
        Assert.AreEqual(1, CountOf(beats, "stack"), "one stack beat");
        Assert.AreEqual(1, CountOf(beats, "shard"), "perfect must drop one shard beat");
        var shot = FirstOf(beats, "shot_released");
        Assert.AreEqual("Perfect", shot.Beat, "beat parsed from detail");
        Assert.AreEqual(0.90f, shot.Power, 0.01f, "power parsed from detail");
        Assert.AreEqual(1, shot.Shot, "shot ordinal parsed from detail");
        Assert.GreaterOrEqual(shot.TimeSec, 0f, "sim timestamp carried");
        var stack = FirstOf(beats, "stack");
        Assert.AreEqual(1, stack.Height, "stack beat carries tower height");
        var shard = FirstOf(beats, "shard");
        Assert.AreEqual(1, shard.Shards, "shard beat carries shard total");
        Assert.AreEqual(1, p.StackHeight, "presenter reads live tower height");
        Assert.AreEqual(1, p.ShardCount, "presenter reads live shard count");
    }
    [Test]
    public void Map3_MissBeatsCarryStabilityAndCollapseResetsTower()
    {
        var p = NewOpened();
        G06ViewCommand[] beats;
        // good shot first (stability 2.0 -&gt; 2.5) so the miss chain shows a mid-band
        // drop before the final negative drop collapses the tower
        p.BeginHold();
        p.Advance(HoldToGood);
        var goodBeat = p.ReleaseShot(out beats);
        Assert.AreEqual(ArrowRushSim.Beat.Good, goodBeat, "setup shot must be good");
        p.Advance(0.30f);
        int guards = 0;
```

```
        while (p.Sim.CollapseCount == 0)
        {
            p.BeginHold();
            p.Advance(0.05f); // low power = miss
            p.ReleaseShot(out beats);
            if (++guards &gt; 20) { Assert.Fail("collapse never triggered on miss spam"); }
        }
        var dropped = FirstOf(beats, "arrow_dropped");
        Assert.IsNotNull(dropped, "miss must emit an arrow_dropped beat");
        Assert.Less(dropped.Stability, 0f, "final arrow_dropped carries post-drain negative stability");
        var collapse = FirstOf(beats, "collapse");
        Assert.IsNotNull(collapse, "stability drain must emit collapse");
        Assert.AreEqual(1, collapse.Collapse, "collapse beat carries its ordinal");
        Assert.AreEqual(0, p.StackHeight, "collapse resets the live tower read");
    }
    [Test]
    public void Map4_ReviveOfferAndAcceptedDrainInOrder()
    {
        var p = NewOpened();
        var rng = new Random(7);
        int guards = 0;
        G06ViewCommand[] beats = new G06ViewCommand[0];
        while (p.Phase != ArrowRushSim.Phase.Failed)
        {
            p.BeginHold();
            p.Advance(0.15f + (float)rng.NextDouble() * 0.10f); // pure miss spam
            p.ReleaseShot(out beats);
            if (++guards &gt; 500) { Assert.Fail("failed state never reached"); }
        }
        var offer = FirstOf(beats, "revive_offer");
        Assert.IsNotNull(offer, "fail release batch must carry revive_offer");
        Assert.AreEqual(ArrowRushSim.Phase.Failed, p.Phase, "presenter reads failed phase");
        var accepted = p.AcceptRevive();
        Assert.AreEqual(1, CountOf(accepted, "revive_accepted"), "revive drains accepted beat");
        Assert.AreEqual(ArrowRushSim.Phase.Playing, p.Phase, "revive restores play phase");
    }
    [Test]
    public void Map5_FirstWowFiresOnceAtHeightThree()
    {
        var p = NewOpened();
        G06ViewCommand[] beats;
        p.BeginHold();
        p.Advance(HoldToPerfect);
        p.ReleaseShot(out beats);
        p.Advance(0.30f);
        Assert.IsNull(FirstOf(beats, "first_wow"), "no wow below height 3");
        for (int i = 0; i &lt; 2; i++)
        {
            p.BeginHold();
            p.Advance(HoldToPerfect);
```

```
                p.ReleaseShot(out beats);
                p.Advance(0.30f);
            }
            var wow = FirstOf(beats, "first_wow");
            Assert.IsNotNull(wow, "third stack must fire first_wow");
            Assert.AreEqual(3, wow.Height, "wow beat carries height 3");
            p.BeginHold();
            p.Advance(HoldToPerfect);
            p.ReleaseShot(out beats);
            Assert.IsNull(FirstOf(beats, "first_wow"), "first_wow must fire exactly once per run");
        }
        [Test]
        public void Map6_ArtStemConventionIsG06Prefix()
        {
            Assert.AreEqual("g06_feather_gold", G06BoardPresenter.ArtStem("feather_gold"));
            Assert.AreEqual(string.Empty, G06BoardPresenter.ArtStem(null), "null id guards empty");
            Assert.AreEqual(string.Empty, G06BoardPresenter.ArtStem(""), "empty id guards empty");
        }
        [Test]
        public void Map7_StabilityFractionClampedToBand()
        {
            var p = NewOpened();
            float initial = p.StabilityFraction;
            Assert.Greater(initial, 0f, "opening stability reads positive");
            Assert.LessOrEqual(initial, 1f, "fraction never exceeds the band top");
            // good shot raises stability (2.0 -&gt; 2.5), then a miss bites (2.5 -&gt; 0.5, no collapse)
            p.BeginHold();
            p.Advance(HoldToGood);
            G06ViewCommand[] beats;
            var beat = p.ReleaseShot(out beats);
            Assert.AreEqual(ArrowRushSim.Beat.Good, beat, "0.672 power must classify good");
            Assert.GreaterOrEqual(p.StabilityFraction, initial, "good shot must not lower stability");
            p.BeginHold();
            p.Advance(0.05f);
            var missBeat = p.ReleaseShot(out beats);
            Assert.AreEqual(ArrowRushSim.Beat.Miss, missBeat, "0.12 power must classify miss");
            Assert.Less(p.StabilityFraction, 0.5f * 0.99f, "a miss must lower the stability fraction");
        }
    }
}
// G06RedlineCheckTests.cs
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using NUnit.Framework;
using Sy.G06;
using UnityEngine;
namespace Sy.G06.Tests
{
```

```
/// &lt;summary&gt;
/// Red line 11 runtime assertion gate (task S2-T006, GAP-C, G03RedlineCheckTests
/// paradigm). Three documented new-player pacings (fast / median / slow) wrap the
/// scripted ten-second opening replay plus the first own hold-release shot; every
/// beat gets a modelled experienced second, the per-tier timeline is written to
/// Logs/g06_redline11_report.txt, and thresholds come from docs/04 table v1:
/// hook frame + subtitle inside T+0-3s, first core op + first stack feedback
/// inside the 10s window (feedback lag &amp;lt;= 1s), three stacks + first wow
/// signal (first_wow banner) inside 10s.
/// Pacing constants are DESIGN ESTIMATES documented here - not device
/// measurements; the full free-play pace model stays the S4 GAP review slot.
/// Boot is batchmode-instantaneous by design; real cold start = G4 gate.
/// The opening replay runs on the sim virtual clock rendered at ReplayRate
/// (G06MainView: the 2.075s virtual opening in ~0.52s experienced); replay
/// beats convert with the same constant, free play runs at 1x. The G06 core
/// op is hold-and-release, so the first own op pays the hold duration
/// (HoldToPerfectSec) after the touch decision. Per T006 acceptance ("gate
/// green OR honest red timeline"), hard NUnit thresholds run on the fast
/// tier while median/slow verdicts land verbatim in the report - never
/// silenced.
/// &lt;/summary&gt;
public class G06RedlineCheckTests
{
    // ---- virtual-clock pacing tiers (design-estimated new-player constants) ----
    // First-session shape: hook caption + subtitle render at t=0, the scripted
    // opening replay plays itself out (~0.52s experienced, input gated), then
    // the player holds and releases the first own shot. Costs modelled per tier:
    //   HookRead  - seconds until attention lands on the hook subtitle
    //   BoardScan - post-replay tower scan before the first own touch
    //   Tap       - decision+action cost of the first own touch
    sealed class Pacing
    {
        public readonly string Name;
        public readonly float HookRead;
        public readonly float BoardScan;
        public readonly float Tap;
        public Pacing(string name, float hookRead, float boardScan, float tap)
        {
            Name = name; HookRead = hookRead; BoardScan = boardScan; Tap = tap;
        }
    }
    static readonly Pacing Fast = new Pacing("fast", 0.8f, 0.3f, 0.3f);
    static readonly Pacing Median = new Pacing("median", 1.6f, 0.8f, 0.5f);
    static readonly Pacing Slow = new Pacing("slow", 2.6f, 1.6f, 0.8f);
    // Opening replay speed parity with G06MainView.ReplayRate; free play at 1x.
    const float ReplayRate = 4f;
    // Scripted opening virtual length: guide beat + 3 x (hold + celebration gap).
    const float OpeningVirtualSec = G06HookFlow.OpeningBeatSec
        + G06HookFlow.OpeningShotCount
          * (G06HookFlow.HoldToPerfectSec + G06HookFlow.CelebrationGapSec);
```

```
const float HookBudgetSec = 3f;         // docs/04: T+0-3s hook frame + subtitle
const float FirstOpBudgetSec = 10f;    // docs/04: first core op inside the 10s window
const float FeedbackLagBudgetSec = 1f; // docs/04: stack feedback lag &lt;= 1s
const float WowBudgetSec = 10f;        // docs/04: first wow signal &lt;= 10s
const int StacksInWindowRequired = 3;   // docs/04: three stacks inside 10s
static string ReportPath =&gt;
    Path.Combine(Application.dataPath, "..", "Logs", "g06_redline11_report.txt");
/// &lt;summary&gt;
/// Every beat the red-line table needs, captured from the real production
/// chain (presenter opening replay + first own shot). Replay beats convert
/// to experienced seconds via ReplayRate; the own feedback lag is measured
/// from the sim itself (free play runs at 1x there).
/// &lt;/summary&gt;
sealed class Timeline
{
    public float CaptionSec;         // scripted caption beat, experienced
    public float GuideSec;           // scripted guide beat, experienced
    public float ScriptedOpSec;      // scripted first shot release, experienced
    public float ScriptedStackSec;   // scripted first stack, experienced
    public float ScriptedLag;        // scripted release -&gt; stack lag, experienced
    public int ScriptedStacksInWindow;
    public float WowSec;             // first_wow banner beat, experienced
    public float ReplayEndSec;       // input unlock, experienced
    public float OwnHoldSec;         // own hold duration at 1x (0.375)
    public float OwnStackLag;        // measured own release -&gt; stack lag
    public int OwnStackCount;         // stacks earned by the first own shot
    public string[] Kinds;           // full command kind set (structure probe)
}
static Timeline CaptureTimeline()
{
    var presenter = new G06BoardPresenter();
    var commands = presenter.RunOpening();
    Assert.IsNotEmpty(commands, "opening must render beats");
    var t = new Timeline
    {
        Kinds = commands.Select(c =&gt; c.Kind).Distinct().OrderBy(k =&gt; k, StringComparer.Ordinal).ToArray()
    };
    var caption = commands.FirstOrDefault(c =&gt; c.Kind == "caption");
    Assert.NotNull(caption, "opening carries the hook caption beat");
    t.CaptionSec = (float)(caption.TimeSec / ReplayRate);
    var guide = commands.FirstOrDefault(c =&gt; c.Kind == "guide");
    Assert.NotNull(guide, "opening carries the first-op guide beat");
    t.GuideSec = (float)(guide.TimeSec / ReplayRate);
    var released = commands.FirstOrDefault(c =&gt; c.Kind == "shot_released");
    Assert.NotNull(released, "opening carries the scripted shot release beat");
    t.ScriptedOpSec = (float)(released.TimeSec / ReplayRate);
    var stacks = commands.Where(c =&gt; c.Kind == "stack").ToArray();
    Assert.GreaterOrEqual(stacks.Length, StacksInWindowRequired,
        "the scripted opening must stack the E4 three towers");
    t.ScriptedStackSec = (float)(stacks[0].TimeSec / ReplayRate);
```

```
        t.ScriptedLag = t.ScriptedStackSec - t.ScriptedOpSec;
        t.ScriptedStacksInWindow = stacks
            .Count(s =&gt; s.TimeSec / ReplayRate &lt;= FirstOpBudgetSec);
        var wow = commands.FirstOrDefault(c =&gt; c.Kind == "first_wow");
        Assert.NotNull(wow, "first_wow banner beat must fire on the height-3 tower");
        t.WowSec = (float)(wow.TimeSec / ReplayRate);
        t.ReplayEndSec = OpeningVirtualSec / ReplayRate;
        Assert.AreEqual(G06HookFlow.OpeningShotCount, presenter.StackHeight,
            "the replay hands over a height-3 real tower (real progress, not a cutscene)");
        // First own shot after the replay: hold to the sweet spot, release,
        // the stack must land the same tick (the tower keeps growing).
        t.OwnHoldSec = G06HookFlow.HoldToPerfectSec;
        presenter.BeginHold();
        presenter.Advance(G06HookFlow.HoldToPerfectSec);
        ArrowRushSim.Beat verdict = presenter.ReleaseShot(out var own);
        Assert.AreEqual(ArrowRushSim.Beat.Perfect, verdict,
            "the modelled first own shot holds to the perfect band");
        var ownReleased = own.FirstOrDefault(c =&gt; c.Kind == "shot_released");
        var ownLanded = own.FirstOrDefault(c =&gt; c.Kind == "arrow_landed");
        var ownStack = own.FirstOrDefault(c =&gt; c.Kind == "stack");
        Assert.NotNull(ownReleased, "the first own shot renders a release beat");
        Assert.NotNull(ownLanded, "the first own shot renders a landing beat");
        Assert.NotNull(ownStack, "the first own shot must stack (tower keeps growing)");
        t.OwnStackCount = own.Count(c =&gt; c.Kind == "stack");
        t.OwnStackLag = (float)(ownStack.TimeSec - ownReleased.TimeSec);
        return t;
    }
    /// &lt;summary&gt;
    /// Modelled experienced clock for one pacing tier: scripted beats keep
    /// their replay-converted seconds, the player pays HookRead / BoardScan /
    /// Tap around the non-interactive replay, and the hold-and-release core
    /// op pays the hold duration before the release beat.
    /// &lt;/summary&gt;
    static Dictionary&lt;string, float&gt; ModelTier(Timeline t, Pacing p)
    {
        var m = new Dictionary&lt;string, float&gt;
        {
            ["hook_hit"] = p.HookRead,
            ["caption_seen"] = t.CaptionSec,
            ["guide_seen"] = t.GuideSec,
            ["first_op_seen"] = t.ScriptedOpSec,
            ["first_stack_seen"] = t.ScriptedStackSec,
            ["scripted_stack_lag"] = t.ScriptedLag,
            ["wow_seen"] = t.WowSec,
            ["input_unlock"] = t.ReplayEndSec,
            ["first_own_touch"] = t.ReplayEndSec + p.BoardScan + p.Tap,
            ["own_hold"] = t.OwnHoldSec,
            ["first_own_op"] = t.ReplayEndSec + p.BoardScan + p.Tap + t.OwnHoldSec,
            ["own_stack_lag"] = t.OwnStackLag,
        };
```

```
    m["first_own_stack"] = m["first_own_op"] + t.OwnStackLag;
    m["stacks_in_10s"] = t.ScriptedStacksInWindow
        + (m["first_own_stack"] &lt;= FirstOpBudgetSec ? t.OwnStackCount : 0);
    return m;
}
static List&lt;string&gt; BuildReportRows(Pacing p, Dictionary&lt;string, float&gt; t)
{
    var rows = new List&lt;string&gt;
    {
        "tier=" + p.Name,
        "  hook_hit          t=" + t["hook_hit"].ToString("0.00") + " (attention lands on the subtitle)",
        "  caption_seen      t=" + t["caption_seen"].ToString("0.00") + " (subtitle on the hook frame)",
        "  guide_seen         t=" + t["guide_seen"].ToString("0.00") + " (first-op guide prompt)",
        "  first_op_seen     t=" + t["first_op_seen"].ToString("0.00") + " (scripted shot release)",
        "  first_stack_seen  t=" + t["first_stack_seen"].ToString("0.00")
            + " (lag " + t["scripted_stack_lag"].ToString("0.00") + "s)",
        "  wow_seen          t=" + t["wow_seen"].ToString("0.00") + " (first_wow banner)",
        "  input_unlock      t=" + t["input_unlock"].ToString("0.00") + " (replay end)",
        "  first_own_touch   t=" + t["first_own_touch"].ToString("0.00"),
        "  own_hold          t=" + t["own_hold"].ToString("0.00") + " (hold to the perfect band)",
        "  first_own_op      t=" + t["first_own_op"].ToString("0.00") + " (release beat)",
        "  first_own_stack   t=" + t["first_own_stack"].ToString("0.00")
            + " (lag " + t["own_stack_lag"].ToString("0.00") + "s)",
        "  stacks_in_10s     =" + t["stacks_in_10s"].ToString("0"),
    };
    string v1 = t["hook_hit"] &lt;= HookBudgetSec ? "GREEN" : "RED";
    string v2 = t["first_own_op"] &lt;= FirstOpBudgetSec ? "GREEN" : "RED";
    string v3 = t["own_stack_lag"] &lt;= FeedbackLagBudgetSec
              &amp;&amp; t["scripted_stack_lag"] &lt;= FeedbackLagBudgetSec ? "GREEN" : "RED";
    string v4 = t["stacks_in_10s"] &gt;= StacksInWindowRequired ? "GREEN" : "RED";
    string v5 = t["wow_seen"] &lt;= WowBudgetSec ? "GREEN" : "RED";
    rows.Add("  VERDICT hook&lt;=" + HookBudgetSec + "s: " + v1
        + " | first_own_op&lt;=" + FirstOpBudgetSec + "s: " + v2
        + " | feedback_lag&lt;=" + FeedbackLagBudgetSec + "s: " + v3
        + " | stacks&gt;=" + StacksInWindowRequired + "in10s: " + v4
        + " | wow&lt;=" + WowBudgetSec + "s: " + v5);
    return rows;
}
[Test]
public void FastPacing_HookAndScriptedBeats_WithinWindows()
{
    var t = ModelTier(CaptureTimeline(), Fast);
    Assert.LessOrEqual(t["hook_hit"], HookBudgetSec,
        "hook attention must land inside the T+0-3s window, modelled=" + t["hook_hit"]);
    Assert.LessOrEqual(t["caption_seen"], HookBudgetSec,
        "hook subtitle must be on the frame inside 3s, modelled=" + t["caption_seen"]);
    Assert.LessOrEqual(t["guide_seen"], HookBudgetSec,
        "the first-op guide must land inside the 3s window, modelled=" + t["guide_seen"]);
    Assert.LessOrEqual(t["first_op_seen"], FirstOpBudgetSec,
        "the scripted first core op must land inside the 10s window");
```

```
    Assert.LessOrEqual(t["scripted_stack_lag"], FeedbackLagBudgetSec,
        "the scripted stack feedback must land &lt;=1s after the release, measured="
        + t["scripted_stack_lag"]);
}
[Test]
public void FastPacing_FirstOwnOpAndFeedback_WithinBudget()
{
    var t = ModelTier(CaptureTimeline(), Fast);
    Assert.LessOrEqual(t["first_own_op"], FirstOpBudgetSec,
        "the first own release must land inside the 10s window, modelled=" + t["first_own_op"]);
    Assert.LessOrEqual(t["own_stack_lag"], FeedbackLagBudgetSec,
        "the own stack feedback must land &lt;=1s after the own release, measured=" + t["own_stack_lag"]);
    Assert.LessOrEqual(t["first_own_stack"], FirstOpBudgetSec,
        "the first own stack must land inside the 10s window, modelled=" + t["first_own_stack"]);
}
[Test]
public void FastPacing_StacksAndWow_InsideWindow()
{
    var t = ModelTier(CaptureTimeline(), Fast);
    Assert.GreaterOrEqual(t["stacks_in_10s"], StacksInWindowRequired,
        "three stacks must land inside the 10s window, modelled=" + t["stacks_in_10s"]);
    Assert.LessOrEqual(t["wow_seen"], WowBudgetSec,
        "the first_wow banner (first wow signal) must land inside the 10s window, modelled="
        + t["wow_seen"]);
}
[Test]
public void MedianAndSlowPacing_Verdicts_Recorded_Honestly()
{
    // The verdict rows must state the truth as measured: RED when over
    // budget, GREEN when within - never silenced for non-fast tiers.
    foreach (var p in new[] { Median, Slow })
    {
        var t = ModelTier(CaptureTimeline(), p);
        var rows = BuildReportRows(p, t);
        string verdict = rows.First(r =&gt; r.Contains("VERDICT"));
        StringAssert.Contains(
            t["hook_hit"] &lt;= HookBudgetSec ? "hook&lt;=3s: GREEN" : "hook&lt;=3s: RED", verdict,
            p.Name + " hook verdict must state the measured truth");
        StringAssert.Contains(
            t["first_own_op"] &lt;= FirstOpBudgetSec ? "first_own_op&lt;=10s: GREEN" : "first_own_op&lt;=10s: RE
            verdict, p.Name + " first_own_op verdict must state the measured truth");
        StringAssert.Contains(
            t["stacks_in_10s"] &gt;= StacksInWindowRequired ? "stacks&gt;=3in10s: GREEN" : "stacks&gt;=3in10s: R
            verdict, p.Name + " stacks verdict must state the measured truth");
    }
}
[Test]
public void ReportFile_CoversAllTiers_Timeline()
{
    var all = new List&lt;string&gt;
```

```
        {
            "# g06 redline11 runtime timeline report (S2-T006) - modelled seconds, design-estimate pacing",
            "# thresholds: docs/04 red-line-11 table v1; boot batchmode-instant (real cold start = G4)",
            "generated=" + DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
        };
        foreach (var p in new[] { Fast, Median, Slow })
        {
            var t = ModelTier(CaptureTimeline(), p);
            all.AddRange(BuildReportRows(p, t));
            Assert.Greater(t["first_own_touch"], t["input_unlock"],
                p.Name + " first own touch must come after the replay unlock");
            Assert.Greater(t["first_own_op"], t["first_own_touch"],
                p.Name + " own release must come after the own touch (hold-and-release op)");
            Assert.GreaterOrEqual(t["first_own_stack"], t["first_own_op"],
                p.Name + " own stack must land with or after the own release");
        }
        string dir = Path.GetDirectoryName(ReportPath);
        if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
        File.WriteAllLines(ReportPath, all);
        Assert.IsTrue(File.Exists(ReportPath), "report file must exist");
        string text = File.ReadAllText(ReportPath);
        foreach (var tier in new[] { "tier=fast", "tier=median", "tier=slow" })
        {
            StringAssert.Contains(tier, text, "report must contain " + tier);
        }
    }
    [Test]
    public void Flow_CommandKinds_BusinessOnly_NoTeaching_Login_OrAnnounce()
    {
        // Red line 11: no white-screen tutorial, no login wall, no announcement
        // popup - the command stream itself must stay a pure business flow,
        // and the replay unlock must lead straight into a stacking tower.
        var t = CaptureTimeline();
        var businessKinds = new HashSet&lt;string&gt;
        {
            "caption", "guide", "shot_released", "arrow_split", "arrow_landed",
            "stack", "split_stack", "arrow_dropped", "shard", "collapse",
            "revive_offer", "revive_accepted", "first_wow", "resync"
        };
        foreach (var kind in t.Kinds)
        {
            Assert.IsTrue(businessKinds.Contains(kind),
                "command kind '" + kind + "' must be a business beat");
            foreach (var banned in new[] { "teach", "tutorial", "login", "announce", "popup", "notice" })
            {
                Assert.IsFalse(kind.ToLowerInvariant().Contains(banned),
                    "no " + banned + " beat may exist in the flow (kind=" + kind + ")");
            }
        }
        Assert.GreaterOrEqual(t.OwnStackCount, 1,
```

```
                "the first own shot after the replay must stack - unlock leads straight into play");
        }
        [Test]
        public void FeedbackContract_CoreClipsUnderOneSecond_LeitmotifUnderHookLine()
        {
            // First-core-op feedback latency contract: every core feedback clip
            // completes inside the 1s latency budget (same-tick wiring, G06Sfx
            // per beat). The leitmotif is the brand hook, not a core-op clip: its
            // own red line is the &lt;=3s hummable signature, asserted present here.
            string[] coreIds =
            {
                G06Sfx.Whoosh, G06Sfx.Thunk, G06Sfx.CollapseCrack,
                G06Sfx.ChainDingId(0), G06Sfx.ChainDingId(G06Sfx.DingMaxStep)
            };
            foreach (var id in coreIds)
            {
                float[] samples = G06Sfx.Synth(id);
                Assert.IsNotNull(samples, "clip " + id + " must synth");
                Assert.IsNotEmpty(samples, "clip " + id + " must have samples");
                float seconds = samples.Length / (float)G06Sfx.SampleRate;
                Assert.LessOrEqual(seconds, FeedbackLagBudgetSec,
                    "clip " + id + " must complete within the 1s latency budget");
            }
            float[] hook = G06Sfx.Synth(G06Sfx.Leitmotif);
            Assert.IsNotEmpty(hook, "leitmotif must have samples");
            float hookSeconds = hook.Length / (float)G06Sfx.SampleRate;
            Assert.LessOrEqual(hookSeconds, 3f, "leitmotif must stay inside its 3s hook line");
        }
    }
}
// G06SaveTests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Core;
using Sy.G06;
using UnityEngine;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// Profile persistence tests (task S2-T001, G02SaveTests paradigm): single-key
    /// JSON round-trip through the Sy.Core save service, the four-state lifecycle,
    /// corruption degradation, version-migration skeleton and the red line 12
    /// zero-server audit (local PlayerPrefs is the only authority).
    /// &lt;/summary&gt;
    public class G06SaveTests
    {
        const string TestSlot = "g06_test_save";
        [TearDown]
        public void TearDown()
```

```
{
    PlayerPrefs.DeleteKey(SaveService.Prefix + TestSlot);
}
static G06Profile SampleProfile()
{
    return new G06Profile
    {
        bestHeight = 42,
        shards = 999,
        ownedSkins = new List&lt;string&gt; { "f01", "f03" },
        equippedSkin = "f03",
    };
}
[Test]
public void RoundTrip_RestoresHeightShardsSkinsAndEquipped()
{
    var save = new SaveService();
    G06SaveSystem.Persist(save, SampleProfile(), TestSlot);
    var loaded = new G06Profile();
    var status = G06SaveSystem.LoadInto(save, loaded, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.Loaded));
    Assert.That(loaded.bestHeight, Is.EqualTo(42));
    Assert.That(loaded.shards, Is.EqualTo(999));
    Assert.That(loaded.ownedSkins, Is.EqualTo(new List&lt;string&gt; { "f01", "f03" }));
    Assert.That(loaded.equippedSkin, Is.EqualTo("f03"));
}
[Test]
public void EmptySlot_YieldsNewGameWithFreshProfile()
{
    var save = new SaveService();
    var profile = SampleProfile(); // stale caller state must not survive a new game
    var status = G06SaveSystem.LoadInto(save, profile, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.NewGame));
    Assert.That(profile.bestHeight, Is.EqualTo(0));
    Assert.That(profile.shards, Is.EqualTo(0));
    Assert.That(profile.ownedSkins.Count, Is.EqualTo(0));
    Assert.That(profile.equippedSkin, Is.EqualTo(""));
}
[Test]
public void CorruptedSlot_DegradesToFreshProfileWithoutThrowing()
{
    PlayerPrefs.SetString(SaveService.Prefix + TestSlot, "{corrupted!!! not json");
    var save = new SaveService();
    var profile = SampleProfile();
    var status = G06SaveSystem.LoadInto(save, profile, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.CorruptedReset));
    Assert.That(profile.bestHeight, Is.EqualTo(0));
    Assert.That(profile.shards, Is.EqualTo(0));
    Assert.That(profile.ownedSkins.Count, Is.EqualTo(0));
    Assert.That(profile.equippedSkin, Is.EqualTo(""));
```

```
}
[Test]
public void FutureVersion_ResetsInsteadOfCrashing()
{
    PlayerPrefs.SetString(SaveService.Prefix + TestSlot,
        "{\"g06v\":99,\"best\":1,\"shards\":1,\"skins\":[\"f06\"],\"equipped\":\"f06\"}");
    var save = new SaveService();
    var profile = new G06Profile();
    var status = G06SaveSystem.LoadInto(save, profile, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.CorruptedReset));
    Assert.That(profile.bestHeight, Is.EqualTo(0), "future payload must never leak into the profile");
    Assert.That(profile.shards, Is.EqualTo(0));
    Assert.That(profile.ownedSkins.Count, Is.EqualTo(0));
    Assert.That(profile.equippedSkin, Is.EqualTo(""));
}
[Test]
public void LegacyPayload_MigratesWithDefaultsForMissingFields()
{
    // A pre-versioning payload: only the height key exists, everything else fills defaults.
    PlayerPrefs.SetString(SaveService.Prefix + TestSlot, "{\"best\":17}");
    var save = new SaveService();
    var profile = new G06Profile();
    var status = G06SaveSystem.LoadInto(save, profile, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.Migrated));
    Assert.That(profile.bestHeight, Is.EqualTo(17), "known legacy field must survive the migration");
    Assert.That(profile.shards, Is.EqualTo(0), "missing fields fill defaults");
    Assert.That(profile.ownedSkins.Count, Is.EqualTo(0));
    Assert.That(profile.equippedSkin, Is.EqualTo(""));
}
[Test]
public void ServiceThrow_DegradesToCorruptedReset()
{
    var save = new ThrowingSaveService();
    var profile = SampleProfile();
    var status = G06SaveSystem.LoadInto(save, profile, TestSlot);
    Assert.That(status, Is.EqualTo(G06LoadStatus.CorruptedReset));
    Assert.That(profile.bestHeight, Is.EqualTo(0));
    Assert.That(profile.equippedSkin, Is.EqualTo(""));
}
[Test]
public void NullGuards_PersistSkipsAndLoadThrows()
{
    Assert.DoesNotThrow(() =&gt; G06SaveSystem.Persist(null, SampleProfile(), TestSlot));
    Assert.Throws&lt;ArgumentNullException&gt;(() =&gt; G06SaveSystem.LoadInto(null, new G06Profile(), TestSlot)
    Assert.Throws&lt;ArgumentNullException&gt;(() =&gt; G06SaveSystem.LoadInto(new SaveService(), null, TestSlot
    Assert.Throws&lt;ArgumentNullException&gt;(() =&gt; G06SaveSystem.Serialize(null));
}
class ThrowingSaveService : ISaveService
{
    public void Save(string slot, string payloadJson) { throw new InvalidOperationException("boom"); }
```

```
            public string Load(string slot) { throw new InvalidOperationException("boom"); }
            public SyncResult CloudSync() { throw new InvalidOperationException("boom"); }
        }
    }
}
// G06SfxTests.cs
using System;
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// Determinism + X028 discipline tests for the procedural sfx (task S1-T005):
    /// synthesis is pure math (no Random, no files) so every render is identical;
    /// the signature hook fits the 3s hummable window and contains both of its
    /// tones; the combo ladder climbs a semitone per step and caps at one octave;
    /// the whoosh/thunk axes stay spectrally distinct; the collapse crack reads as
    /// multiple wood snaps, not one thud.
    /// &lt;/summary&gt;
    public class G06SfxTests
    {
        static int ZeroCrossings(float[] buf)
        {
            int n = 0;
            for (int i = 1; i &lt; buf.Length; i++)
                if ((buf[i - 1] &lt;= 0f &amp;&amp; buf[i] &gt; 0f) || (buf[i - 1] &gt;= 0f &amp;&amp; buf[i] &lt; 0f))
            return n;
        }
        [Test]
        public void Synth_Is_Deterministic_And_Bounded_For_Every_Clip()
        {
            string[] ids =
            {
                G06Sfx.Whoosh, G06Sfx.Thunk, G06Sfx.Leitmotif, G06Sfx.CollapseCrack,
                G06Sfx.ChainDingId(0), G06Sfx.ChainDingId(5), G06Sfx.ChainDingId(12),
            };
            foreach (var id in ids)
            {
                var a = G06Sfx.Synth(id);
                var b = G06Sfx.Synth(id);
                Assert.Greater(a.Length, 0, "clip must synthesize: " + id);
                Assert.AreEqual(a.Length, b.Length, "the same recipe keeps one length: " + id);
                double peak = 0.0;
                for (int i = 0; i &lt; a.Length; i++)
                {
                    Assert.AreEqual(a[i], b[i], "synthesis must be deterministic (no Random): " + id);
                    Assert.IsFalse(float.IsNaN(a[i]) || float.IsInfinity(a[i]), "samples must be finite: " + id);
                    Assert.LessOrEqual(Math.Abs(a[i]), 1f, "normalized samples stay inside [-1,1]: " + id);
                    peak = Math.Max(peak, Math.Abs(a[i]));
                }
```

```
            Assert.GreaterOrEqual(peak, 0.5, "normalization keeps the loudness family tight: " + id);
        }
    }
    [Test]
    public void Signature_Hook_Is_A_Real_Phrase_Inside_The_Three_Second_Window()
    {
        var hook = G06Sfx.Synth(G06Sfx.Leitmotif);
        double sec = hook.Length / (double)G06Sfx.SampleRate;
        Assert.LessOrEqual(sec, 3.0, "X028: the hook must be hummable within 3s");
        Assert.GreaterOrEqual(sec, 0.30, "the hook must be a real phrase, not a blip");
        // the signature is literally the whoosh-thunk pair: both tones inside
        var whoosh = G06Sfx.Synth(G06Sfx.Whoosh);
        var thunk = G06Sfx.Synth(G06Sfx.Thunk);
        Assert.GreaterOrEqual(hook.Length, whoosh.Length + thunk.Length,
            "the leitmotif must contain both signature tones");
    }
    [Test]
    public void Whoosh_And_Thunk_Stay_Spectrally_Distinct_Axes()
    {
        // X028 mutual-exclusion contour check: the glide (high, fast) and the
        // knock (low, heavy) must never blur into one instrument.
        var whoosh = G06Sfx.Synth(G06Sfx.Whoosh);
        var thunk = G06Sfx.Synth(G06Sfx.Thunk);
        double whooshRate = ZeroCrossings(whoosh) / (whoosh.Length / (double)G06Sfx.SampleRate);
        double thunkRate = ZeroCrossings(thunk) / (thunk.Length / (double)G06Sfx.SampleRate);
        Assert.Greater(whooshRate, thunkRate * 2.0,
            "the whoosh (high glide) must cross zero far faster than the thunk (low knock)");
    }
    [Test]
    public void Ding_Ladder_Climbs_A_Semitone_Per_Step_And_Caps_At_One_Octave()
    {
        int prev = -1;
        for (int step = 0; step &lt;= G06Sfx.DingMaxStep; step++)
        {
            int crossings = ZeroCrossings(G06Sfx.Synth(G06Sfx.ChainDingId(step)));
            Assert.Greater(crossings, prev,
                "the combo ladder must ascend (brighter ding per streak step): step " + step);
            prev = crossings;
        }
        // steps beyond the octave hold the peak; negatives fold to the floor
        Assert.AreEqual(ZeroCrossings(G06Sfx.Synth(G06Sfx.ChainDingId(G06Sfx.DingMaxStep + 5))),
                        ZeroCrossings(G06Sfx.Synth(G06Sfx.ChainDingId(G06Sfx.DingMaxStep))),
                        "the ladder caps at one octave and holds the peak");
        Assert.AreEqual(ZeroCrossings(G06Sfx.Synth(G06Sfx.ChainDingId(-3))),
                        ZeroCrossings(G06Sfx.Synth(G06Sfx.ChainDingId(0))),
                        "negative steps fold to the floor step");
        // every step is the same recipe shape: identical buffer lengths
        Assert.AreEqual(G06Sfx.Synth(G06Sfx.ChainDingId(0)).Length,
                        G06Sfx.Synth(G06Sfx.ChainDingId(G06Sfx.DingMaxStep)).Length,
                        "the same recipe keeps one length across the ladder");
```

```
        }
        [Test]
        public void Collapse_Crack_Reads_As_Multiple_Wood_Snaps()
        {
            var crack = G06Sfx.Synth(G06Sfx.CollapseCrack);
            double sec = crack.Length / (double)G06Sfx.SampleRate;
            Assert.LessOrEqual(sec, 1.0, "the collapse crack stays a short burst");
            // count attack fronts: near-silence erupting into a strong transient.
            // ka-la = several snaps, not one thud.
            int fronts = 0;
            bool armed = true;
            for (int i = 0; i &lt; crack.Length; i++)
            {
                float a = Math.Abs(crack[i]);
                if (armed &amp;&amp; a &gt; 0.45f) { fronts++; armed = false; }
                else if (!armed &amp;&amp; a &lt; 0.06f) armed = true;
            }
            Assert.GreaterOrEqual(fronts, 3,
                "the collapse must read as multiple wood snaps (found " + fronts + " attack fronts)");
        }
    }
}
// G06SplitPresenterTests.cs
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// S2-T003 slice B: presenter split-command mapping gate. The drain layer must
    /// hand the fork beats to the view as typed commands (arrow_split carries the
    /// deterministic fork anchor splits/fraction; split_stack carries height + the
    /// fork arrow ordinal), in the frozen sim order shot_released -&gt; arrow_split -&gt;
    /// arrow_landed -&gt; stack/split_stack rows. The resync fallback is a presentation
    /// safety net only: with every sim kind mapped 1:1 the temporary resync state is
    /// retired - a full playthrough surface must drain ZERO resync commands.
    /// &lt;/summary&gt;
    public class G06SplitPresenterTests
    {
        const float HoldToPerfect = 0.375f; // 0.375 * 2.4 = 0.90 power, inside the perfect band
        const float HoldToGood = 0.28f;     // 0.28 * 2.4 = 0.672 power, good band
        const float HoldToTop = 0.42f;      // 1.008 -&gt; clamps at the meter top (full power)
        static G06BoardPresenter NewOpened()
        {
            var p = new G06BoardPresenter();
            p.Open();
            p.Advance(0.05f);
            p.MaybeShowGuide();
            return p;
        }
        static int CountOf(G06ViewCommand[] beats, string kind)
```

```
{
    int n = 0;
    for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) n++;
    return n;
}
static int IndexOf(G06ViewCommand[] beats, string kind)
{
    for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) return i;
    return -1;
}
[Test]
public void SplitMap1_FullPowerShotDrainsTypedSplitBeatsInFrozenOrder()
{
    var p = NewOpened();
    p.BeginHold();
    p.Advance(HoldToTop);
    var beat = p.ReleaseShot(out var beats);
    Assert.AreEqual(ArrowRushSim.Beat.Good, beat, "meter-top verdict stays Good (vocabulary frozen)");
    Assert.AreEqual(1, CountOf(beats, "arrow_split"), "exactly one arrow_split beat");
    var fork = beats[IndexOf(beats, "arrow_split")];
    Assert.AreEqual(3, fork.Splits, "fork beat carries the split count");
    Assert.AreEqual(0.50f, fork.Fraction, 1e-4f, "fork beat carries the deterministic height fraction");
    Assert.AreEqual(1, CountOf(beats, "stack"), "main arrow keeps one stack beat");
    Assert.AreEqual(2, CountOf(beats, "split_stack"), "fork arrows land as two split_stack beats");
    var first = beats[IndexOf(beats, "split_stack")];
    var second = beats[IndexOf(beats, "split_stack") + 1];
    Assert.AreEqual(2, first.ArrowIndex, "first fork row carries arrow=2/3 ordinal");
    Assert.AreEqual(3, second.ArrowIndex, "second fork row carries arrow=3/3 ordinal");
    Assert.AreEqual(2, first.Height, "fork rows land at heights 2 and 3");
    Assert.AreEqual(3, second.Height);
    int iShot = IndexOf(beats, "shot_released");
    int iFork = IndexOf(beats, "arrow_split");
    int iLand = IndexOf(beats, "arrow_landed");
    int iStack = IndexOf(beats, "stack");
    Assert.Less(iShot, iFork, "fork happens after release");
    Assert.Less(iFork, iLand, "fork happens strictly mid-flight, before landing");
    Assert.Less(iLand, iStack, "stack rows follow the landing");
    Assert.Less(iStack, IndexOf(beats, "split_stack"),
        "main row lands first, fork rows after it");
    Assert.AreEqual(3, p.StackHeight, "presenter reads the 3-row tower");
}
[Test]
public void SplitMap2_ResyncRetiredAcrossFullEventSurface()
{
    var p = NewOpened();
    var drained = new System.Collections.Generic.List&lt;G06ViewCommand&gt;();
    System.Action&lt;G06ViewCommand[]&gt; collect = b =&gt; drained.AddRange(b);
    // one beat of every non-fork verdict: perfect / good / full-power split
    p.BeginHold(); p.Advance(HoldToPerfect); p.ReleaseShot(out var b1); collect(b1);
    p.Advance(0.30f);
```

```
            p.BeginHold(); p.Advance(HoldToGood); p.ReleaseShot(out var b2); collect(b2);
            p.Advance(0.30f);
            p.BeginHold(); p.Advance(HoldToTop); p.ReleaseShot(out var b3); collect(b3);
            p.Advance(0.30f);
            // miss chain to the fail state, then revive: closes the surface
            int guards = 0;
            while (p.Phase != ArrowRushSim.Phase.Failed)
            {
                p.BeginHold();
                p.Advance(0.05f);
                p.ReleaseShot(out var bm); collect(bm);
                if (++guards &gt; 500) { Assert.Fail("failed state never reached"); }
            }
            collect(p.AcceptRevive());
            Assert.Greater(drained.Count, 20, "the flow must exercise a rich command surface");
            for (int i = 0; i &lt; drained.Count; i++)
                Assert.AreNotEqual("resync", drained[i].Kind,
                    "resync state retired: every sim kind must map 1:1 (offender detail: " + drained[i].Detail + ")");
        }
        [Test]
        public void SplitMap3_PlainShotsNeverLeakSplitCommands()
        {
            var p = NewOpened();
            float[] holds = { HoldToPerfect, HoldToGood, 0.05f };
            foreach (float hold in holds)
            {
                p.BeginHold();
                p.Advance(hold);
                p.ReleaseShot(out var beats);
                Assert.AreEqual(0, CountOf(beats, "arrow_split"),
                    "non-full-power shot must never drain arrow_split");
                Assert.AreEqual(0, CountOf(beats, "split_stack"),
                    "non-full-power shot must never drain split_stack");
            }
            Assert.AreEqual(2, p.StackHeight, "perfect+good stack two rows, miss stacks none");
        }
    }
}
// G06SplitTests.cs
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// S2-T003 slice A: full-power split mechanic (pitch mechanism 2, GAP-B first half).
    /// Bot-empirical pinning for the three GAP-B tuning questions:
    ///   trigger - full power is the meter-top clamp frame (charge 1.0), the only
    ///             value above PerfectMax reachable at any frame rate;
    ///   split   - 3 arrows fork mid-flight at SplitHeightFraction (slice B renders);
    ///   landing - 3 stack rows from one shot, good-tier stability (+0.5, never the
```

```
///             perfect bonus), zero shards (perfect-only economy stays frozen).
/// The E4 13-assert tree is untouched: perfect/good/miss resolutions stay identical.
/// &lt;/summary&gt;
public class G06SplitTests
{
    const float HoldToPerfect = 0.375f; // 0.375 * 2.4 = 0.90 power, inside the perfect band
    const float HoldToGreed = 0.4125f;  // 0.4125 * 2.4 = 0.99 power, band-top slide zone
    const float HoldToTop = 0.42f;      // 1.008 -&gt; clamps at the meter top (full power)
    static ArrowRushSim NewOpenedSim()
    {
        var sim = new ArrowRushSim();
        sim.Open();
        return sim;
    }
    static void Shoot(ArrowRushSim sim, float hold)
    {
        sim.BeginHold();
        sim.AdvanceClock(hold);
        sim.ReleaseShot();
    }
    [Test]
    public void Split1_TriggerIsMeterTopClampFrame()
    {
        var sim = NewOpenedSim();
        Shoot(sim, HoldToTop);
        Assert.AreEqual(1, sim.SplitShots, "meter-top release must fire exactly one split");
        Assert.AreEqual(1, sim.CountEvents("arrow_split"), "arrow_split event must fire once");
    }
    [Test]
    public void Split2_SplitEventCarriesDeterministicForkAnchor()
    {
        var sim = NewOpenedSim();
        Shoot(sim, HoldToTop);
        string detail = null;
        foreach (var e in sim.Events)
            if (e.Kind == "arrow_split") { detail = e.Detail; break; }
        Assert.IsTrue(detail != null, "arrow_split event missing");
        Assert.AreEqual("splits=3 fraction=0.50", detail,
            "fork anchor detail must stay deterministic (slice B View consumes it)");
    }
    [Test]
    public void Split3_BandTopSlideZoneStaysPlainGood()
    {
        var sim = NewOpenedSim();
        sim.BeginHold();
        sim.AdvanceClock(HoldToGreed);
        var beat = sim.ReleaseShot();
        Assert.AreEqual(ArrowRushSim.Beat.Good, beat, "0.99 verdict stays Good (vocabulary frozen)");
        Assert.AreEqual(0, sim.SplitShots, "below the clamp a shot must never split");
        Assert.AreEqual(1, sim.StackHeight, "plain good stacks exactly one row");
```

```
    Assert.AreEqual(0, sim.CountEvents("split_stack"));
}
[Test]
public void Split4_PerfectBandNeverSplitsAndKeepsShardEconomy()
{
    var sim = NewOpenedSim();
    Shoot(sim, HoldToPerfect);
    Assert.AreEqual(ArrowRushSim.Beat.Perfect, ArrowRushSim.Classify(0.90f));
    Assert.AreEqual(0, sim.SplitShots, "perfect release must never split");
    Assert.AreEqual(1, sim.ShardCount, "perfect still owns the shard economy");
    Assert.AreEqual(0, sim.CountEvents("arrow_split"));
}
[Test]
public void Split5_LandingResolutionThreeStackRowsOneShot()
{
    var sim = NewOpenedSim();
    Shoot(sim, HoldToTop);
    Assert.AreEqual(3, sim.StackHeight, "one full-power shot must stack 3 rows");
    Assert.AreEqual(1, sim.CountEvents("stack_signal"), "main arrow keeps the stack_signal kind");
    Assert.AreEqual(2, sim.CountEvents("split_stack"), "fork arrows land as split_stack rows");
    Assert.AreEqual(0, sim.CountEvents("arrow_dropped"), "full power never drops stability");
    Assert.AreEqual(0, sim.CountEvents("shard_drop"), "full power never drops shards");
    Assert.AreEqual(1, sim.Shots, "the split is one shot, not three");
}
[Test]
public void Split6_GoodTierStabilityNeverPerfectBonus()
{
    var sim = NewOpenedSim();
    float before = sim.Stability; // fresh sim opens at 2.0
    Shoot(sim, HoldToTop);
    Assert.AreEqual(before + 0.5f, sim.Stability, 1e-6f,
        "full power grants the good-tier +0.5 only, never the perfect +1");
    Shoot(sim, HoldToTop);
    Assert.AreEqual(before + 1.0f, sim.Stability, 1e-6f,
        "second split also adds +0.5 (2.0 -&gt; 2.5 -&gt; 3.0)");
}
[Test]
public void Split7_FirstWowCanFireFromASingleFullPowerShot()
{
    var sim = NewOpenedSim();
    Shoot(sim, HoldToTop);
    Assert.AreEqual(1, sim.CountEvents("first_wow"),
        "a 3-row tower from one shot is a legitimate first-wow moment");
    Assert.AreEqual(3, sim.StackHeight);
}
[Test]
public void Split8_BotBarePlaySplitStrategyLoopIsDeterministic()
{
    var sim = NewOpenedSim();
    // deterministic strategy cycle: top-clamp split / perfect / 0.99 greed / miss
```

```
            float[] cycle = { HoldToTop, HoldToPerfect, HoldToGreed, 0.05f };
            const int cycles = 25;
            for (int i = 0; i &lt; cycles; i++)
                for (int k = 0; k &lt; cycle.Length; k++)
                    Shoot(sim, cycle[k]);
            Assert.AreEqual(cycles * cycle.Length, sim.Shots, "bot must fire all 100 shots");
            Assert.AreEqual(cycles, sim.SplitShots, "exactly one split per cycle");
            Assert.AreEqual(125, sim.StackHeight,
                "height must grow +5 per cycle (3+1+1; the miss drains stability only)");
            Assert.AreEqual(0, sim.CollapseCount,
                "steady-state stability cycle (2.0-&gt;2.5-&gt;3.5-&gt;4.0-&gt;2.0) never collapses");
            Assert.AreEqual(cycles, sim.ShardCount, "shards stay perfect-only");
        }
    }
}
// G06WindPresenterTests.cs
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// S2-T004 slice B: presenter wind-command mapping gate. The drain layer must
    /// hand the hourglass beats to the view as typed commands: wind_shift carries
    /// the direction (+1 tail / -1 head) alternating flip by flip, wind_catch
    /// carries the post-shield stability typed out of the sim detail and lands
    /// strictly after the verdict beats, and the catch fires only inside the
    /// armed window after a head-&gt;tail flip. With wind off (the default) the full
    /// play surface drains zero wind commands - the E4 13-assert tree never sees
    /// the layer (opt-in default OFF, task S2-T004 slice A).
    /// &lt;/summary&gt;
    public class G06WindPresenterTests
    {
        const float HoldToPerfect = 0.375f; // 0.375 * 2.4 = 0.90 power, inside the perfect band
        static G06BoardPresenter NewOpened()
        {
            var p = new G06BoardPresenter();
            p.Open();
            p.Advance(0.05f);
            p.MaybeShowGuide();
            return p;
        }
        static int CountOf(G06ViewCommand[] beats, string kind)
        {
            int n = 0;
            for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) n++;
            return n;
        }
        static int IndexOf(G06ViewCommand[] beats, string kind)
        {
            for (int i = 0; i &lt; beats.Length; i++) if (beats[i].Kind == kind) return i;
```

```
        return -1;
    }
    [Test]
    public void WindMap1_ShiftBeatsDrainTypedDirAlternating()
    {
        var p = NewOpened();
        p.EnableWind();
        var beats = p.Advance(10f);   // first flip on the period boundary: tail -&gt; head
        Assert.AreEqual(1, CountOf(beats, "wind_shift"), "exactly one shift at the first boundary");
        var first = beats[IndexOf(beats, "wind_shift")];
        Assert.AreEqual(-1, first.WindDir, "first flip drains head wind (the glass opened tail)");
        Assert.AreEqual(-1, p.WindDirection, "read face agrees with the drained dir");
        beats = p.Advance(10f);       // second flip: head -&gt; tail (arms the catch window)
        var second = beats[IndexOf(beats, "wind_shift")];
        Assert.AreEqual(1, second.WindDir, "second flip drains tail wind");
        Assert.AreEqual(1, p.WindDirection);
        Assert.LessOrEqual(p.WindPhaseRemaining, ArrowRushSim.WindPeriodSeconds,
            "phase remaining stays inside one period");
    }
    [Test]
    public void WindMap2_CatchInsideWindowDrainsTypedStabilityAfterVerdicts()
    {
        var p = NewOpened();
        p.EnableWind();
        p.Advance(20f);               // flips: t=10 head, t=20 tail (window armed)
        p.BeginHold();
        p.Advance(HoldToPerfect);
        var beat = p.ReleaseShot(out var beats);
        Assert.AreEqual(ArrowRushSim.Beat.Perfect, beat);
        Assert.AreEqual(1, CountOf(beats, "wind_catch"), "release inside the window catches the gust");
        var caught = beats[IndexOf(beats, "wind_catch")];
        Assert.AreEqual(4.0f, caught.Stability, 1e-4f,
            "typed stability: start 2 + perfect 1 + shield 1 = 4");
        Assert.Less(IndexOf(beats, "shot_released"), IndexOf(beats, "wind_catch"),
            "the shield lands after the verdict beats");
        Assert.Greater(p.StabilityFraction, 0.5f, "stability read face reflects the shield");
    }
    [Test]
    public void WindMap3_ReleaseOutsideWindowDrainsNoCatch()
    {
        var p = NewOpened();
        p.EnableWind();
        p.Advance(20f);               // tail flip armed the window
        p.Advance(2f);                // the 1.2s window has lapsed
        p.BeginHold();
        p.Advance(HoldToPerfect);
        p.ReleaseShot(out var beats);
        Assert.AreEqual(0, CountOf(beats, "wind_catch"),
            "a late release never catches the gust");
        Assert.AreEqual(0, CountOf(beats, "wind_shift"),
```

```
                "no new flip inside the 2s gap");
        }
        [Test]
        public void WindMap4_WindOffDrainsZeroWindCommands()
        {
            var p = NewOpened();
            var drained = new System.Collections.Generic.List&lt;G06ViewCommand&gt;();
            drained.AddRange(p.Advance(25f));   // two flips would have fired with wind on
            p.BeginHold();
            p.Advance(HoldToPerfect);
            p.ReleaseShot(out var b1); drained.AddRange(b1);
            drained.AddRange(p.Advance(5f));
            drained.AddRange(p.Advance(5f));
            Assert.IsFalse(p.WindEnabled, "default stays off (E4 frozen surface)");
            for (int i = 0; i &lt; drained.Count; i++)
            {
                Assert.AreNotEqual("wind_shift", drained[i].Kind, "no shift beats with wind off");
                Assert.AreNotEqual("wind_catch", drained[i].Kind, "no catch beats with wind off");
            }
        }
    }
}
// G06WindTests.cs
using NUnit.Framework;
using Sy.G06;
namespace Sy.G06.Tests
{
    /// &lt;summary&gt;
    /// S2-T004 slice A: wind hourglass (pitch mechanism 3, GAP-B second half).
    /// Bot-empirical pinning for the three GAP-B tuning questions:
    ///   period   - 10s from the pitch, validated as a rhythm event (12 flips over a
    ///              120s+ bare-play loop, a whole shot cluster fits inside one phase);
    ///   bonus    - catch = +1 stability shield on any beat; a caught miss nets -1
    ///              (absorbs half the miss penalty, never rewrites verdicts or rows);
    ///   headwind - sweep x1.5 from the band sweep {1.25 no perceivable effect,
    ///              1.5 exactly one verdict tier down for the fixed-hold bot,
    ///              1.75+ miss cascade}: harder but fair.
    /// The E4 13-assert tree stays frozen: wind is opt-in, default OFF (Wind1).
    /// &lt;/summary&gt;
    public class G06WindTests
    {
        const float HoldToPerfect = 0.375f; // 0.375 * 2.4 = 0.90 power, inside the perfect band
        static ArrowRushSim NewOpenedSim()
        {
            var sim = new ArrowRushSim();
            sim.Open();
            return sim;
        }
        static void Advance(ArrowRushSim sim, float seconds, float step)
        {
```

```
    int n = (int)(seconds / step + 0.5f);
    for (int i = 0; i &lt; n; i++) sim.AdvanceClock(step);
}
[Test]
public void Wind1_DefaultOffKeepsE4ParitySurfaceFrozen()
{
    var sim = NewOpenedSim();
    sim.BeginHold();
    sim.AdvanceClock(0.375f);
    Assert.AreEqual(0.90f, sim.Charge, 1e-4f, "sweep must stay the E4 anchor while wind is off");
    sim.ReleaseShot();
    Advance(sim, 120f, 0.5f);
    Assert.AreEqual(0, sim.CountEvents("wind_shift"), "no flip events without opt-in");
    Assert.AreEqual(0, sim.CountEvents("wind_catch"), "no catch events without opt-in");
    Assert.IsFalse(sim.WindEnabled);
}
[Test]
public void Wind2_FlipsAreDeterministicAndAlternate()
{
    var sim = NewOpenedSim();
    sim.EnableWind();
    Advance(sim, 120f, 0.5f);
    Assert.AreEqual(12, sim.CountEvents("wind_shift"), "120s must flip exactly 12 times (10s period)");
    string expected = "dir=head"; // hourglass opens tail, the first flip crosses into head
    foreach (var e in sim.Events)
        if (e.Kind == "wind_shift")
        {
            Assert.AreEqual(expected, e.Detail, "flip detail must alternate head/tail");
            expected = expected == "dir=head" ? "dir=tail" : "dir=head";
        }
    Assert.AreEqual(10.0f, sim.FirstEventTime("wind_shift"), 1e-4f,
        "first flip lands on the period boundary");
    Assert.AreEqual(1, sim.WindDirection, "an even flip count returns to tail");
}
[Test]
public void Wind3_HeadwindSweepMultiplierIsExact()
{
    var sim = NewOpenedSim();
    sim.EnableWind();
    sim.BeginHold();
    sim.AdvanceClock(0.375f);
    Assert.AreEqual(0.90f, sim.Charge, 1e-4f, "tail phase keeps the E4 sweep anchor");
    sim.ReleaseShot();
    Advance(sim, 9.7f, 0.1f); // crosses the first flip into head
    Assert.AreEqual(-1, sim.WindDirection, "first flip crosses into head");
    sim.BeginHold();
    sim.AdvanceClock(0.25f);
    Assert.AreEqual(0.90f, sim.Charge, 1e-4f,
        "head reaches 0.90 in 0.25s = 0.25 * 2.4 * 1.5 (the pinned multiplier)");
}
```

```
    [Test]
    public void Wind4_CatchWindowArmsOnTailwardFlipOnly()
    {
        var sim = NewOpenedSim();
        sim.EnableWind();
        // opening tail phase: the enable moment is a phase start, not a reversal
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        Assert.AreEqual(0, sim.CountEvents("wind_catch"), "enable must not arm the window");
        Advance(sim, 20.5f, 0.1f); // flips: t~10 head, t~20 tail (armed)
        Assert.AreEqual(2, sim.CountEvents("wind_shift"));
        Assert.AreEqual(1, sim.WindDirection, "the second flip restores tail");
        Assert.IsTrue(sim.WindCatchOpen, "window open right after the tail-ward flip");
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        Assert.AreEqual(1, sim.CountEvents("wind_catch"), "in-window release catches the gust");
        Advance(sim, 1.5f, 0.1f);
        Assert.IsFalse(sim.WindCatchOpen, "window closes after 1.2s");
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        Assert.AreEqual(1, sim.CountEvents("wind_catch"), "one catch per tail-ward flip");
    }
    [Test]
    public void Wind5_HeadPhaseReleaseNeverCatches()
    {
        var sim = NewOpenedSim();
        sim.EnableWind();
        Advance(sim, 10.5f, 0.1f); // first flip crossed: head phase
        Assert.AreEqual(-1, sim.WindDirection);
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        Assert.AreEqual(0, sim.CountEvents("wind_catch"),
            "head wind has no gust to catch (window is tail-ward flips only)");
    }
    [Test]
    public void Wind6_CaughtMissNetsMinusOne()
    {
        var sim = NewOpenedSim();
        // pre-charge stability with two perfects so the miss cannot trip a collapse
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
        Assert.AreEqual(4.0f, sim.Stability, 1e-6f, "two perfects clamp stability at 4.0");
        sim.EnableWind();
        Advance(sim, 20.2f, 0.1f); // two flips: head, then tail (armed)
        Assert.AreEqual(1, sim.WindDirection);
        Assert.IsTrue(sim.WindCatchOpen);
        int heightBefore = sim.StackHeight;
        sim.BeginHold(); sim.AdvanceClock(0.05f); var beat = sim.ReleaseShot();
        Assert.AreEqual(ArrowRushSim.Beat.Miss, beat, "the verdict stays a miss (vocabulary frozen)");
        Assert.AreEqual(1, sim.CountEvents("arrow_dropped"));
        Assert.AreEqual(1, sim.CountEvents("wind_catch"), "the gust catch still fires on a miss");
        Assert.AreEqual(3.0f, sim.Stability, 1e-6f,
            "miss -2 + catch +1 nets -1 (4.0 -&gt; 3.0): the shield absorbs half a miss");
        Assert.AreEqual(heightBefore, sim.StackHeight, "catch adds stability, never rows");
```

```
        }
        [Test]
        public void Wind7_PeriodIsARhythmEventAtBotCadence()
        {
            var sim = NewOpenedSim();
            sim.EnableWind();
            // E4 bare-play bot cadence: hold 0.375 + 0.30 celebration gap = 0.675s/shot
            const int shots = 180; // ~121.5s of play
            for (int i = 0; i &lt; shots; i++)
            {
                sim.BeginHold(); sim.AdvanceClock(0.375f); sim.ReleaseShot();
                sim.AdvanceClock(0.30f);
            }
            Assert.AreEqual(shots, sim.Shots);
            int flips = sim.CountEvents("wind_shift");
            Assert.GreaterOrEqual(flips, 10, "a 120s+ loop must flip a dozen times");
            Assert.LessOrEqual(flips, 13, "flips must stay on the 10s period, not churn");
            Assert.GreaterOrEqual(ArrowRushSim.WindPeriodSeconds / 0.675f, 3f,
                "one phase must hold a whole shot cluster (rhythm event, not churn)");
        }
        [Test]
        public void Wind8_HeadwindBandSweepPinsOneTierDown()
        {
            // analytic sweep-mirror math for the fixed-hold bot (0.375s hold):
            // raw = 0.375 * ChargeSweepSpeed * mul; the meter clamps at 1.0 and reflects.
            float Mirror(float raw) =&gt; raw &lt;= 1f ? raw : 2f - raw;
            Assert.AreEqual(ArrowRushSim.Beat.Perfect, ArrowRushSim.Classify(Mirror(0.375f * 2.4f * 1.25f)),
                "mul 1.25: 0.875 still perfect - no perceivable difficulty");
            Assert.AreEqual(ArrowRushSim.Beat.Good, ArrowRushSim.Classify(Mirror(0.375f * 2.4f * 1.5f)),
                "mul 1.5: 0.65 good - exactly one verdict tier down (the pin)");
            Assert.AreEqual(ArrowRushSim.Beat.Miss, ArrowRushSim.Classify(Mirror(0.375f * 2.4f * 1.75f)),
                "mul 1.75: 0.425 miss - collapse cascade for the naive bot");
            Assert.AreEqual(ArrowRushSim.Beat.Miss, ArrowRushSim.Classify(Mirror(0.375f * 2.4f * 2.0f)),
                "mul 2.0: 0.2 miss - punishing");
            // in-sim observed value at the pinned mul must match the analytic mirror:
            // frame-stepped 0.375s hold (7x0.05 + 0.025) at speed 3.6 -&gt; 0.73 = Good
            var sim = NewOpenedSim();
            sim.EnableWind();
            Advance(sim, 10.1f, 0.1f); // first flip crossed: head phase
            Assert.AreEqual(-1, sim.WindDirection);
            sim.BeginHold();
            for (int i = 0; i &lt; 7; i++) sim.AdvanceClock(0.05f);
            sim.AdvanceClock(0.025f);
            var beat = sim.ReleaseShot();
            Assert.AreEqual(0.73f, sim.Charge, 1e-3f, "frame-stepped naive hold lands at 0.73 in headwind");
            Assert.AreEqual(ArrowRushSim.Beat.Good, beat,
                "naive same-hold bot drops exactly one verdict tier in headwind (harder but fair)");
        }
    }
}
```